

**Synligare utveckling av inbyggda fordonssystem –
visualiserad kravhantering och samverkan**

**Visibler Development of Embedded Automotive Systems -
Visualised Requirements Management and Collaboration**

Report type	Deliverable D2.1
Report name	Modelling Concepts and Methods
Dissemination level	Public
Status	Released
Version number	1.1
Date of preparation	2015-05-29

Authors

Editor**Urban Ingelsson****E-mail****urban.ingelsson@semcon.com****Authors****Henrik Lönn****Magnus Skoog****Mikael Stolpe****Urban Ingelsson****E-mail****henrik.lonn@volvo.com****magnus.skoog@autoliv.com****mikael.stolpe@arccore.com****urban.ingelsson@semcon.com****The Consortium**

Volvo Technology Corporation AB - Autoliv AB - Arccore AB- Systemite AB - Semcon Sweden AB

Revision chart and history log

Version	Date	Reason
0.1	2014-01-01	Document skeleton
0.2	2014-06-06	Re-structuring
0.3	2014-06-22	Tightening of the material into a report
0.4	2014-06-28	New material added
0.5	2014-06-30	Initial Release
0.6	2014-09-13	Revision towards release
0.7	2014-09-22	Version for review
0.8	2014-09-29	Merged version with review comments
1.0	2014-09-30	Release
1.1	2015-05-29	Updates (place-and-route, diagram exchange, etc)



Table of contents

Authors.....	3
Revision chart and history log	5
Table of contents	7
1 Terminology	11
2 Introduction	13
2.1 Purpose.....	14
2.2 Scope.....	14
2.2.1 View Categories.....	14
2.2.2 Further scope.....	16
2.3 Document Outline	16
3 Information Modelling	17
3.1 Need: Model Distribution	17
3.2 Need: Simplification of Version Control	17
3.3 Need: Model Exchange	17
3.4 Need: Model Separation into Architectural Levels	17
3.5 EAST-ADL is an Example of a Structured Information Model.....	18
3.6 Information Model Structure	18
3.7 Tooling Support	19
3.8 Requirements on Tools used for Exchanging Models.....	21
3.9 Exchange between Models in EAST-ADL and Various Tools.....	22
3.9.1 Example of Exchange of EAST-ADL Models Involving SystemWeaver.....	22
3.9.2 Example of Exchange of EAST-ADL Models Involving Enterprise Architect	23
3.10 Normative and Informative Elements	24
3.11 Diagram Model	25
3.12 Requirements on Tools for Exchange of Diagrams.....	28
4 Manufacturer-Supplier Interaction	29
4.1 Traditional Interaction between OEM and Tier-1.....	29
4.2 Suggested Interaction between OEM and Tier-1	34
5 Metrics	37
5.1 Requirement Validation Progress.....	37
5.2 Requirement Allocation Progress	38
5.3 Realization Progress.....	40
5.4 Verification Progress.....	41
5.5 Safety-related Progress Metrics	42
5.6 Product Metrics.....	44
5.7 Identifying Context	45
6 Views	46
6.1 Generic Tree View	46
6.1.1 Example View	46

6.1.2	<i>Related Modelling Concepts</i>	47
6.2	Viewing Related Model Objects.....	48
6.2.1	<i>Example View</i>	48
6.3	Creating connector elements.....	49
6.3.1	<i>Example View</i>	49
6.3.2	<i>Related modeling Concepts</i>	50
6.4	Compare and Merge.....	50
6.4.1	<i>Example View</i>	50
6.5	Increments of Features.....	51
6.5.1	<i>Example View</i>	51
6.6	Features and Requirements.....	52
6.6.1	<i>Example View</i>	53
6.7	Functions and Requirements.....	53
6.7.1	<i>Example View</i>	53
6.8	Hardware Components and Requirements.....	54
6.8.1	<i>Example View</i>	54
6.9	Allocation.....	55
6.9.1	<i>Example View</i>	55
6.9.2	<i>Related Modelling Concepts</i>	56
6.10	Feature-Based Architecture Views.....	56
6.10.1	<i>Example View</i>	56
6.10.2	<i>Related Modeling Concepts</i>	57
6.11	Requirements vs. Architecture.....	58
6.11.1	<i>Example View</i>	59
6.11.2	<i>Related Modelling Concepts</i>	59
6.12	Components, Ports and Signals.....	59
6.12.1	<i>Example Views</i>	60
6.12.2	<i>Related Modeling Concepts</i>	62
6.13	Item Definition.....	62
6.13.1	<i>Example View</i>	62
6.13.2	<i>Related Modeling Concepts</i>	63
6.14	Functional and Technical Safety Concept.....	64
6.14.1	<i>Example View</i>	65
6.14.2	<i>Modelling Concepts</i>	65
6.15	Error Propagation Modelling.....	65
6.15.1	<i>Example View</i>	66
6.15.2	<i>Related Modeling Concepts</i>	67
6.15.3	<i>Work-Phases when Eliciting Error Propagation Models</i>	68
6.16	Illustrating Metrics.....	69
6.16.1	<i>Example View</i>	69

7	Summary.....	71
8	Bibliography	72
9	Acknowledgements.....	73

1 Terminology

Below is a set of terms used in project Synligare. The list largely excludes meta-model elements from AUTOSAR and EAST-ADL, as these are defined in the respective specification. Most of this terminology was defined in Synligare Deliverable D1.1 [1].

Term	Description
Abstract view	A subset of a system description
AUTOSAR	AUTomotive Open System Architecture
Artifact	A general term for engineering elements that compose a solution
ARXML	AUTOSAR exchange format
Component	A general term for software, hardware and functional components
Concrete view	The information of an abstract view, as it is shown to a software tool user. For example, it can have layout, coloring, etc.
DIA	Development Interface Agreement
EAST-ADL	Automotive Architecture Description Language
EAXML	EAST-ADL exchange format
Element	A general term for parts of a model.
Embedded system	Software and electronics embedded in a product
FROP	Feature Roll Out Plan
Function Point	A measurement of business functionality
GraphML	Graph exchange format
Legacy tools	Tools that already exist in a specific context
Metric	A quantification of aspect(s) of a design, system, project, organization , etc.
OEM	Original Equipment Manufacturer
Realize	Relationship between requirement and system model element
Requirement	A singular documented physical and functional need that a particular design, product or process must be or do
Requirement management	The activities of documenting, analyzing, tracing, prioritizing and agreeing on requirements and then controlling change and communicating to relevant stakeholders.
RFI	Request for Information
RFQ	Request for Quotation
Satisfy	Relationship between requirements
SEooC	Safety Element out of Context
SIL	Safety Integrity Level
Synligare	FFI Project Grant FFI 2013-01296
System Model	A set of elements and relations that represents a system
TRL	Technology Readiness Level, a measure introduced by NASA to characterize technology maturity from 1 (initial concept) to 9 (deployed technology)
Tier-n Supplier	A supplier that provides components directly to an OEM (Tier-1) or Tier-i

	supplier (tier-i+1)
User Stories	Short description of functionality in informal language
View	A representation of a whole system (or aspects thereof) from the perspective of a related set of concerns
Viewpoint	A specification of the conventions for constructing and using a view
WP	Work Package, one of five parts of Project Synligare, e.g. WP2
Work product	The tangible result of a particular methodology or process step
Well-formed	The property of a model that it complies with applicable syntactical rules
XML	Extensible Markup Language

2 Introduction

This document is the report on results of Work Package 2 (WP2) of Project Synligare. This work package and subsequently this report describe conceptual ways to address the needs that were identified in D1.1.

In Synligare Deliverable D1.1 [1], the increasing complexity of specifications for automotive embedded systems was identified as a problem that Project Synligare is to address. The key idea behind Project Synligare is to provide overview and to ease management of system specification by means of defining structured information models for representing system specifications. The structured information models will enable us to keep the system specification in the form of a system model, and define views and metrics as well as algorithms for automatically generating them. The views and metrics will provide the overview of the system model to the engineers developing the embedded system.

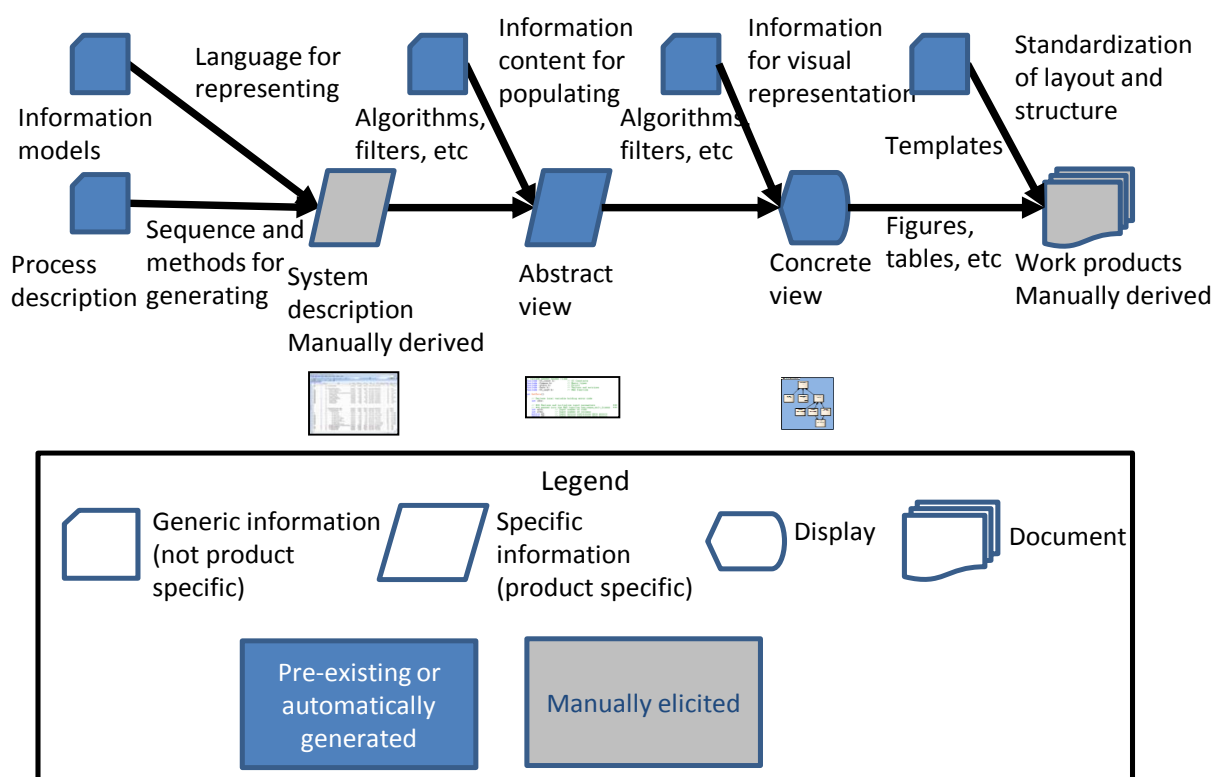


Figure 1. How information models relate to system description and views

It can be seen in Figure 1 how information models, process descriptions, templates and algorithms for generating graphs are not specific to the product. These types of information can be used by an organization to standardize how they conduct their system development projects to ensure compatibility with various tools and quality systems, to gain various benefits. For example, it can be beneficial to purposefully structure information to be easily communicated into a supplier's or a customer's system. **Information models** describe how the system model should be represented, i.e. what components, relationships and attributes there should be. **Process descriptions** describe in what order and by what method to generate the system model as well as criteria for passing milestones in the development project. An **abstract view** is a subset of the system model selected to support a particular activity. A view or **concrete view** contains the same information as the abstract view, but is represented to a stakeholder to give a user-friendly overview over the information to support the activity. There are many aspects of the accessibility and visualization of information that can greatly contribute to understanding of a complex system. Similarly, there are

metrics, i.e. values calculated from the system model to describe progress, performance, quality or other quantifiable aspect of the system. It is expected that **algorithms** or rule sets are required to generate (or draw) the concrete view from the abstract view. However, these algorithms can be generic (they do not need to be system-specific), as long as the abstract view (and the rest of the system model) is represented in accordance with the information model. Typically, the concrete view is a set of graphs, tables and other illustrations. With the help of **templates**, these graphs and tables can be used in **work products** such as the formal documentation of the system under development. It should be noted that describing the system, i.e. eliciting the system description, is a manual process. When information has been entered into the system description as specified in the information models, generation of concrete views can be automated by algorithms that assume information structured according to the information models. The assembly of figures and tables into work products is also a manual process, but is a good example of an activity that is made easier by automatic generation of concrete views.

Defining structured information models involves determining what information carriers (components, relationships and attributes) are required in the various design and verification activities. It should be considered what algorithms are needed to be carried out, like searches and sorting, to arrive at an information model that serves well to represent system specifications. A starting point is to consider the existing specification languages like EAST-ADL. Further, it should be considered how to do management of specifications and requirements in accordance with ISO 26262. The aim is that the outcomes of Project Synligare should be tried on an example system, to validate the structured information models, the views and the metrics.

This chapter introduces WP2 by defining purpose, scope and method.

2.1 Purpose

The work in WP2 will define structured information models that enable communication of system models in collaborative development of automotive embedded systems with the help of views and metrics. In other words, WP2 will develop a representation for system models for communication between OEM and Tier-1 (marked as “information model” in Figure 1), as well as conceptual solutions for visualizing information from the system model (marked as “algorithms” in Figure 1). The conceptual solutions are to be handed on to WP3, for development of corresponding prototype tools. With the help of those prototype tools, the conceptual solutions will be validated with respect to an example system in WP4. The expected effect of having appropriate structured information models for specification and appropriate views and metrics is increased predictability of safety, quality and performance of the specified system. Further it is expected that with a common understanding of how the specification is represented, visualized and measured, communication of specification should involve less misunderstanding. The collaboration aspect shall be considered such that a specification according to the structured information models can be modularized and communicated appropriately.

2.2 Scope

The scope of WP2 is defined by the following categories of views and metrics (that were identified WP1 and reported in D1.1 [1]).

2.2.1 View Categories

Table 1

<i>Topic</i>	<i>Viewpoint</i>	<i>Section in this report</i>	<i>Comment</i>
All Elements	Tree View	6.1	Provides generic hierarchical view of all elements and their attributes

	Properties View	6.2	Relating the attributes of any small set of selected modelling elements.
	Create Connector View	6.3	Tool support for selecting ports and creating modelling elements for the connectors between the ports
	Compare and merge	6.4	Aid in comparing two models, highlighting differences and allowing choices to be made as part of the process of merging two models
Requirements	Clustered on features	6.6	Requirements grouped by which feature they are (indirectly) allocated to
	Clustered on functions	6.7	Requirements grouped by which function they are (indirectly) allocated to
	Clustered on components	6.8	Requirement grouped by which component they are (indirectly) allocated to
	Showing relevant attributes	6.11	Gives a partial discussion on showing attributes, that can be well extended to include all relevant attributes
	Multiple aspects according to selected order and sorting	6.11, 6.13, 6.14	Gives typical applications of selection of requirements allocated to a subsystem or having a particular attribute, which can be extended to encompass other applications of selection and sorting.
Safety	All safety goals for a certain item	6.13	Shows the boundaries of the safety-critical system.
	Functional Safety Concept	6.14	Visualizes what is designed to meet the safety goals, including functional safety requirements and functional modelling elements
	Technical Safety Concept	6.14	Visualizes how to meet the functional safety concept with technical design choices, including technical design modelling elements and hardware safety requirements and software safety requirements
	Safety analysis	6.15	Error propagation modelling and fault tree analysis
System and Software Architecture	All elements realizing a certain feature	6.10	Highlighting the modelling elements in a system model that realize a feature.
	Function allocation to software architecture	6.9	A table with software architectural elements as rows and functions in columns.
	Architecture of distributed software	6.12	Visualization of a software architecture with software components, ports and signals.
	Feature tree	6.5	A variability breakdown of features
Progress	Requirements coverage	6.16	A view for visualizing metrics, such as requirements coverage
	Elements coverage	6.16	A view for visualizing metrics, such as elements coverage

Metric Categories

Topic	Metric	Section in this report	Comment
Product metric	Dependability	5.6	E.g. failure rate
	Impact of system model on production volume, income and cost	5.6	E.g. take rate of a feature configuration
Progress metric	Number of completed analyses / artefacts / reviews	5.1, 5.2, 5.3, 5.5	Provides typical use examples of a metric to follow-up progress.
	Requirements validation	5.1	Comparing a package of validated requirements with a package of not-yet-validated requirements
	Verification progress	5.4	Counting the number of performed verification cases as compared to the full set of verification cases

2.2.2 Further scope

The view categories and metric categories listed above are addressed to ensure that the structured information models that are to be defined can support generation of those views and metrics. Information modelling results are presented in Chapter 3 of this document.

Given the focus on interaction between OEM and Tier-1, the interaction and exchanges between OEM and Tier-1 in a collaborative development project is described in Chapter 4 of this document. The views and metrics that are relevant for the RFQ-process are prioritized to focus on finding solutions that support collaborative system development.

The scope includes ensuring that the structured information models are adequate to specify an automotive embedded system.

2.3 Document Outline

The remainder of this report is structured as follows. Chapter 3 presents information modelling aspects. Chapter 4 discusses the interaction between OEM and Tier-1. Chapter 5 presents metrics as conceptual solutions. Chapter 6 presents views as conceptual solutions. Finally, Chapter 7 summarizes the report and Chapter 8 gives the bibliography.

3 Information Modelling

This chapter reports on the information models used. In order to handle a full scale model we need a model structure that matches the needs. Such needs are:

- Model distribution
- Simplification of version control
- Simplification of model exchange and synchronization
- Model separation into architecture levels.

3.1 Need: Model Distribution

The concept of model distribution concerns the capability to have a model on which several organisations or teams can work in parallel. The models have a common purpose, but different parties work on different parts. This is key when developing subsystems and in collaborative development in which involved organisations may want to withhold some information in the exchanges.

3.2 Need: Simplification of Version Control

Version control is important for exchange of information, to be able to refer to a history of system development. In particular, it is important to be able to determine if the shared information is up to date and synchronized. To make this simpler, a system model should be subject to a structure that allows parts of the model to be separate configuration items. Furthermore, versioning information is useful when searching/filtering information, when showing differences between models, when managing reuse and when planning.

3.3 Need: Model Exchange

These are typical model exchange scenarios:

- A typical scenario for exchanging models is export/import/sync. This means that a model is exported from tool A and imported to tool B. If tool B already contains the model the local model is synchronized during import. This simple scenario does not need any merging.
- A more complex scenario is round-trip. This scenario means that the model is re-imported. The re-imported model replaces the previous model, so merge is not needed.
- The most complex scenario is model merging. In this scenario the user chooses which elements that are merged into the existing model and which parts that are not merged.

3.4 Need: Model Separation into Architectural Levels

Architectural levels are useful for abstracting high-level design from design choices, functional design from implementation details and implementation of logical functions from the details of physical components. It is vital for understanding the system model that the whole system is described on all abstraction levels, so that the design can be followed from concept to implementation. By having the system described on all abstraction levels, issues on a detailed level can be understood on a feature level. In fact, it is state-of-the-art practice to trace requirements from concept to detailed requirements and allocate requirements on feature, function, and implementation levels.

3.5 EAST-ADL is an Example of a Structured Information Model

As an example of a structured information model, this project considers EAST-ADL. Here follows an introduction to EAST-ADL.

According to EAST-ADL models are put into four different architectural levels. This can be seen to meet the need of model separation into architectural levels as is mentioned in Section 3.4.

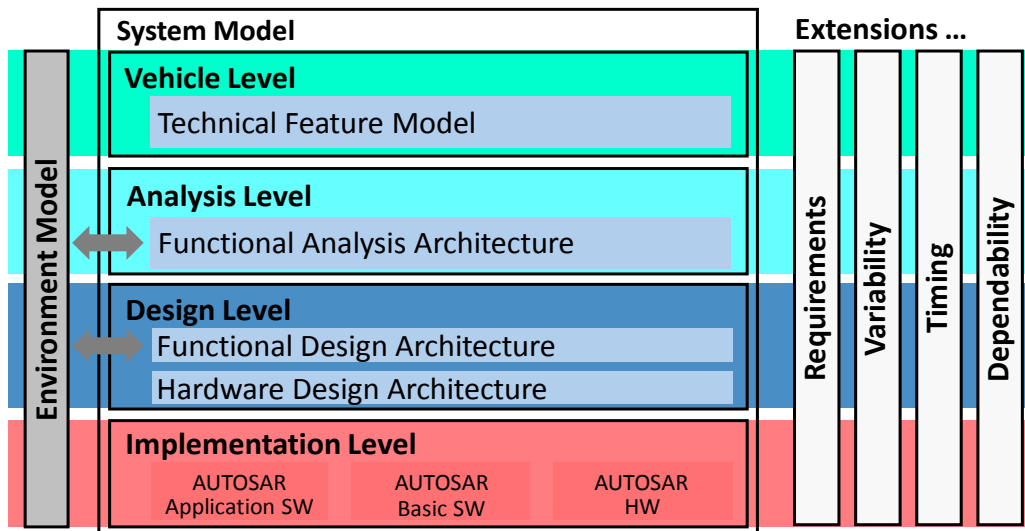


Figure 2: The EAST-ADL's breakdown in abstraction levels (vertically) and in core system model, environment and extensions (horizontally).

The functionality of the vehicle is described at different levels of abstraction and in different parts.

The four abstraction levels covered by the EAST-ADL are:

- **Vehicle Level**
Feature trees characterizing the vehicle content as it is perceived externally.
- **Analysis Level**
Abstract functional architecture defining systems from a functional point of view.
- **Design Level**
Detailed functional architecture allocated to hardware architecture.
- **Implementation Level**
Implementation of the embedded system represented using AUTOSAR elements

EAST-ADL elements do not have a dedicated property for e.g. version, time stamp and status. Instead, User Defined Attributes can be used for this purpose. There is a need to include versioning information in EAST-ADL models. Consequently, EAST-ADL cannot in itself meet the need of simplified version control as is mentioned in Section 3.2.

3.6 Information Model Structure

In Project Synligare, the use of EAST-ADL and other structured information modelling is as follows.

The information is strictly separated into the following parts according to diagram in Figure 3:

- EAST-ADL full system model (package)
- EAST-ADL sub-system model(s) (sub-package)

- In EAST-ADL, sub packages contain the function architecture and HW architecture from sub system perspective
- EAST-ADL Error model(s) (sub-package)
- Diagram models in Diagram-XML. Format is described in Section 3.11.
- AUTOSAR SW models

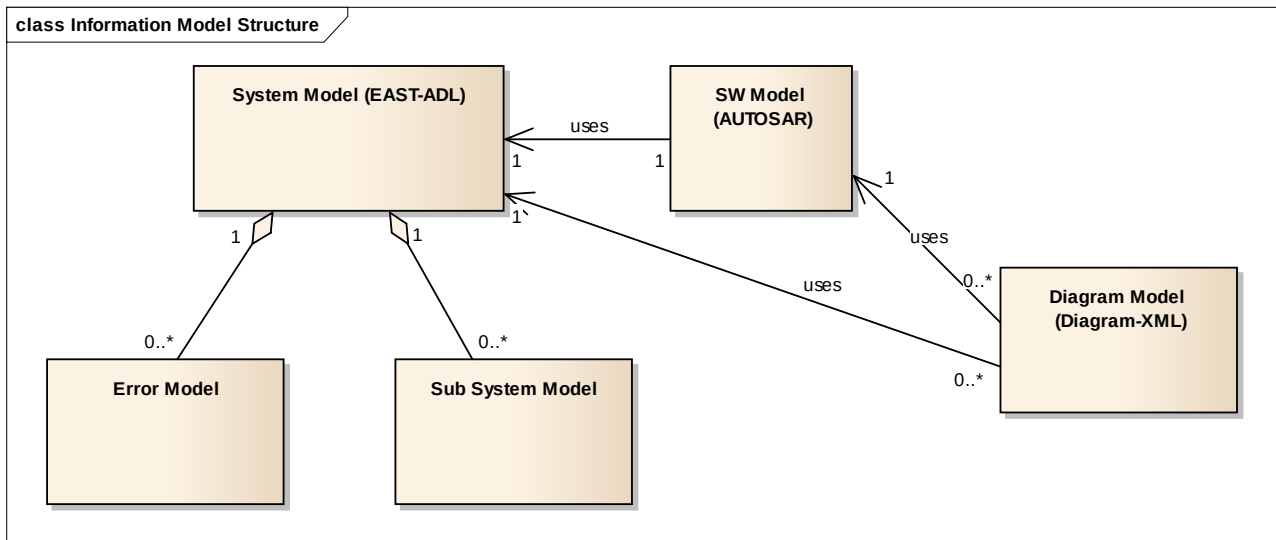


Figure 3: Generic Model Structure

The motivation for the chosen structure is that the system model, error model and the sub-system models shall be possible to handle separately in EAST-ADL. This means that they can be loaded / saved as a separate configuration item. This can be seen as a way to meet the need of model distribution and a way to meet the need of simplification of version control. The separate configuration items can be handled separately and will have their own versioning.

3.7 Tooling Support

The tools considered in Project Synligare are SystemWeaver (SW), EATop, ARTop and Enterprise Architect (EA).

Figure 4 shows the principles for data exchange of information models according to Section 3.6.

In different tools the models are held together in different ways:

- SystemWeaver: Model database
- EATOP: All .eaxml-files that are on the same directory are contained as a complete set of models.
- Enterprise Architect: There is a project file that refers to the individual packages that are implemented as files on the file system.

These various ways of keeping packages together or maintaining a database are considered in the project by working towards a round-trip validation experiment in which models are exchanged between organizations and the three tools named above. Thus Project Synligare will work towards having a working system of information models that withstand the second most complex scenario that is mentioned in Section 3.3.

SystemWeaver and Enterprise Architect have their own practice for version control. However, EATop does not have functionality for version control.

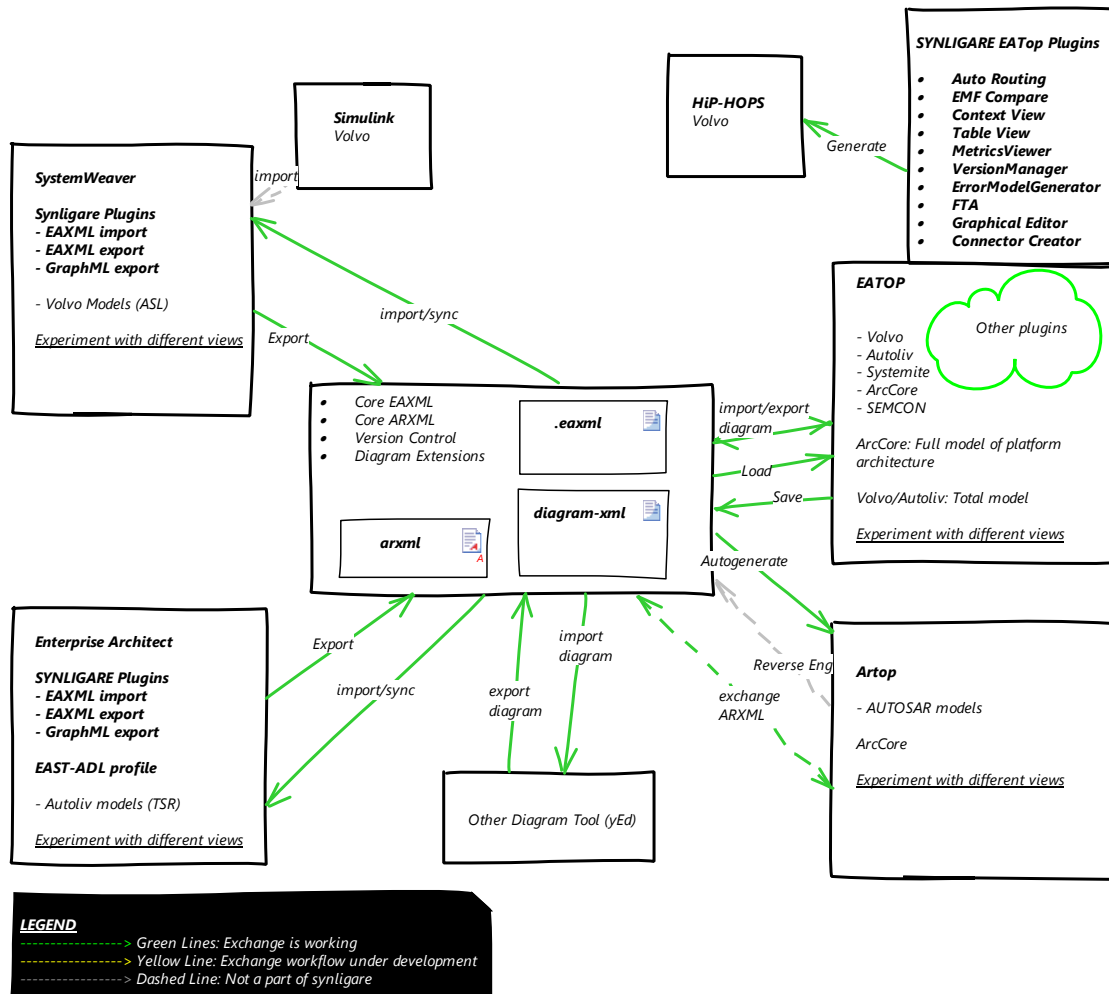


Figure 4. Data exchange model

Figure 4 contains an example. Consider that an OEM, in this case Volvo, would develop an adaptive speed limiter system (ASL) and collaborate with a Tier-1, in this case Autoliv, on developing a traffic sign recognition subsystem (TSR).

Their respective tools contain their own models, stored separately on each part's configuration management tool. The model exchange functionality is implemented by each tool vendor. There is a core model for the system and there is a tool to access it.

This means that the OEM (e.g. Volvo) has the core system model for ASL. It is exported and can be used by Tier-1 (e.g. Autoliv), but it is normally not changed. The Tier-1 can make some updates in the interface and export it back to the OEM, and this can be compared to the core model by the OEM.

The Tier-1 works with the model for the TSR subsystem.

The Tier-1 and the OEM may choose to export only parts of the model. This is fine as long as the requirements, interfaces and the common function are synchronized.

All partners: OEM (Volvo), Tier-1 (Autoliv), Engineering Bureau (Semcon) and Tier-2 (ArcCore, Systemite) can work in EATop with the full model or parts of it. This shows how model distribution can be accomplished to fulfil a need mentioned in Section 3.1.

3.8 Requirements on Tools used for Exchanging Models

To address the challenges involved in exchanging models, such as the challenges are described in Section 3.3, the following will specify requirements on each software tool that is involved in the exchange.

Each exchange tool must be able to handle the following:

- Import of a new model
 - o created in any tool that can produce EAST-ADL models
 - o saved in .eaxml-format (according to the EAST-ADL XML schema eastadl_2-1-12.xsd [2])
- Previously imported models could only be synchronized during import. It shall be possible to do this in one of these methods:
 - o Either each change can be monitored by the user and be individually accepted/rejected.
 - o Alternatively all updates can be shown and the user can do the total import or stop it.

Other requirements:

- The UUID which uniquely identify objects in the core tool should not be changed during import and shall be stored together with the other data. Each element is uniquely identified by its path in a hierarchy of elements and its name in its parent element.
- At synchronization or round-trip an element may have been moved from a package to another or renamed. If it still has the same UUID, the importing tool shall choose to rename or move it instead of re-creating it. This is in order to minimize the updating (and to do the same thing that was intended at the update in the exported tool).
- An export from an imported .eaxml-file shall produce a semantically identical .eaxml-file
- It shall be possible to differentiate between the old model and the updated, but merging is optional
- An object that no longer is a part of the new (imported) model is not removed during synchronization. A warning is printed and it is up to the user to remove manually.
- Regarding parsing of .eaxml-files that do not fully comply with the corresponding XML-schema, the EAST-ADL schema eastadl_2-1-12.xsd [2] is used for validation of the .eaxml-file. If there is an error a warning is printed out, but the model is still imported (for all items that match).
- Where possible, links whose target element is missing should be preserved. On validation, there should be a warning, but the reference to the missing element should be preserved until the element appears in the model or the link is voluntarily deleted or edited.
- The versioning concept should be compatible with established software tools. In particular SystemWeaver, EATop and Enterprise Architect are considered to be established software tools.

Even with the requirements that are listed above, the model exchange will have some imitations:

- No support for variability
- No support for versions (Consequently, the information model structure and the tools considered in Project Synligare do not meet the need of simplification of version control as described in Section 3.2.)

- Import/Synchronization is done manually. This means that the imported model may not match if synchronization is done seldom.
- In this work package, no concept has been developed for supporting error modelling in the tool Enterprise Architect
- Import is prohibited if two object have the same package name or two objects have the same UUID
- Handling of hanging links (links whose target element is missing) may be problematic to solve in Enterprise Architect. Models that relate must all be imported otherwise the links will not be possible to resolve and an export function will miss them. One alternative is to add dummy elements to model in order to make it possible the make the link. Conceptually, such dummy elements could be be replaced when a model is imported that contains them.

3.9 Exchange between Models in EAST-ADL and Various Tools

The OEM and Tier-1 has their own project areas and save the local models by using the local tools file format and local version control systems. The exchange is done via a common project area through export/import to/from .eaxml-files.

3.9.1 Example of Exchange of EAST-ADL Models Involving SystemWeaver

Consider a system model that is developed inside SystemWeaver. When the model is mature it is exported to .eaxml. This model is put into the common project area.

When the Tier-1 has exported a sub-system model it can be imported to SystemWeaver via .eaxml format.

It is also possible to re-import parts of the system model (round-trip).

The workflow is shown in Figure 5.

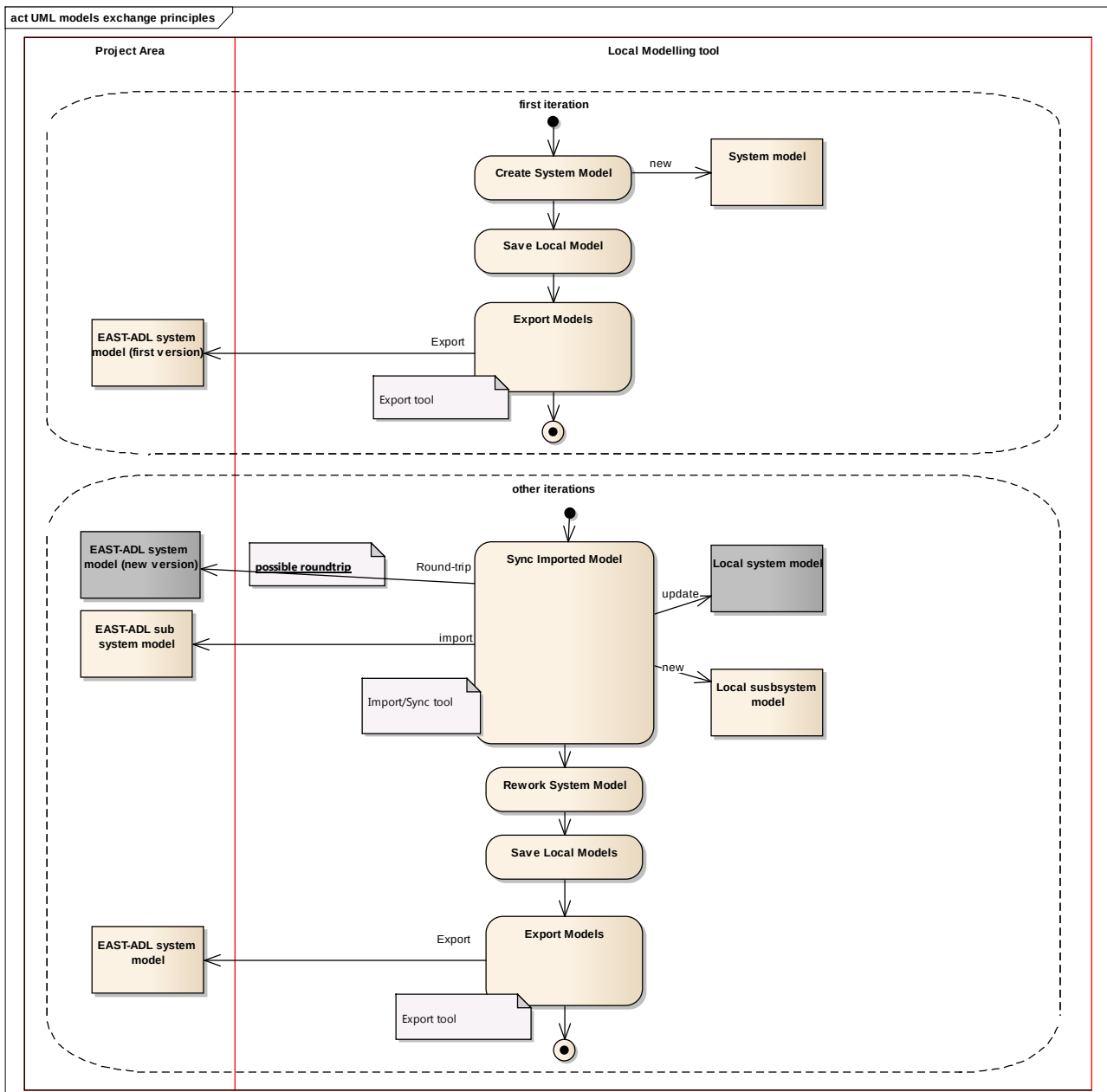


Figure 5. Model Import/Export/Sync/Round-trip from OEM perspective

3.9.2 Example of Exchange of EAST-ADL Models Involving Enterprise Architect

Consider a Tier-1 that has developed a sub-system. The Tier-1 can import a system model from .eaxml format and add its sub-system to the system model.

The sub-system model is developed inside UML tool Enterprise Architect. UML has a lot in common with EAST-ADL. There are a couple of differences that are challenging:

- In EAST-ADL elements are referenced by its package path, but in UML elements are referenced by its UUID.
- In EAST-ADL unresolved relations can exist, but in UML unresolved relations cannot exist.

UML models can be split into configuration items based on packages that can be separately version controlled. Similarly, the EAST-ADL model can be split into several files.

The workflow is shown in Figure 6.

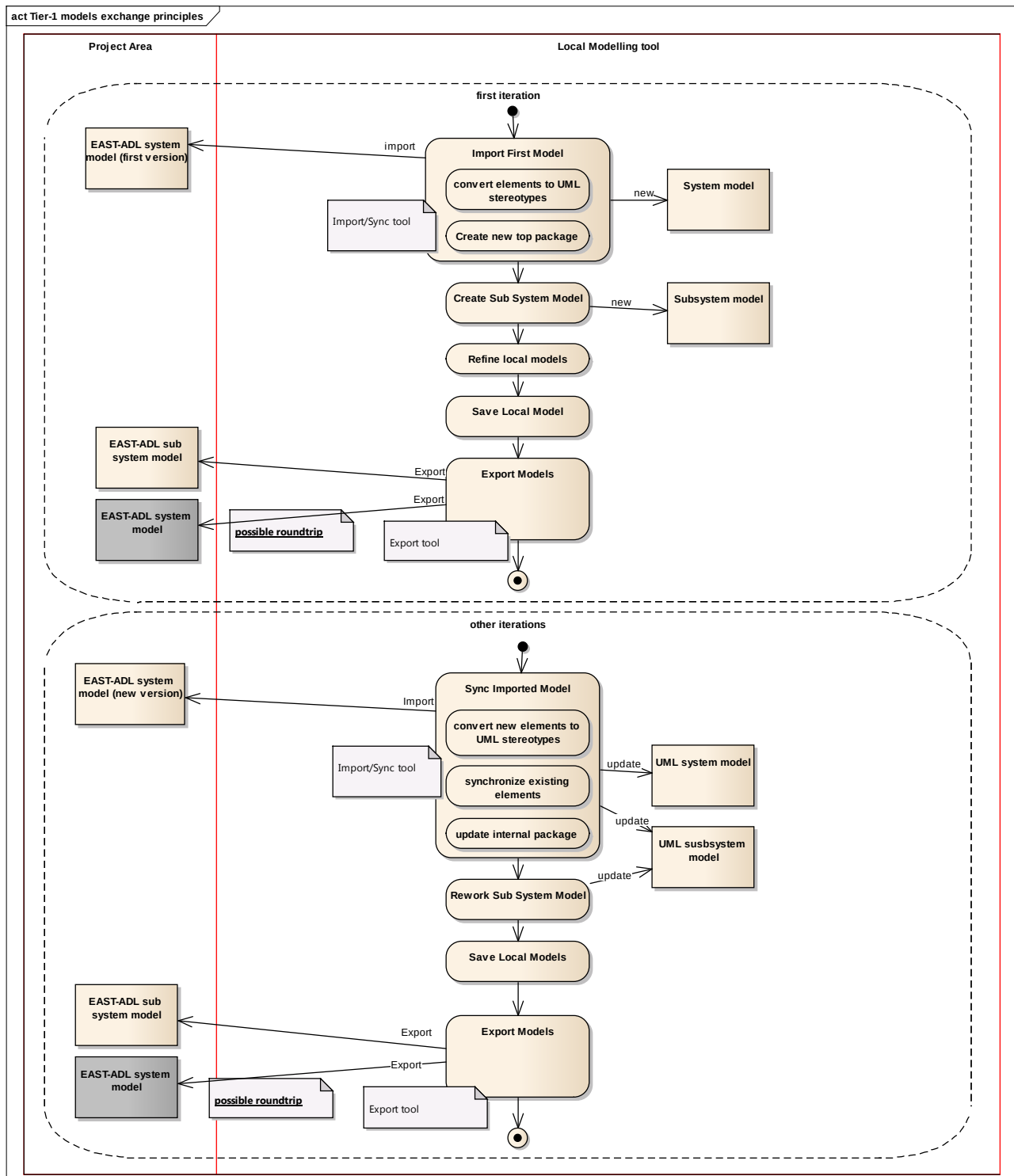


Figure 6. Model Import/Export/Sync/Round-trip from Tier-1 perspective

3.10 Normative and Informative Elements

Another aspect on model exchange is that architectural models can be used to replace textual requirements that describe solutions. In order to distinguish required content in an architectural

model from example or negotiable content, elements can be marked informative or normative with a user defined attribute. The semantics for such user defined attribute should be that

- Default property is “Normative” meaning that the element must be included according to the given specification
- Children in a containment or type-prototype hierarchy inherit the property from its parent
- The element’s property overrides default property and inherited property

Figure 7 shows modelling elements and relations pertaining to User Defined Attributes, which can be used to mark other modelling elements as normative or informative.

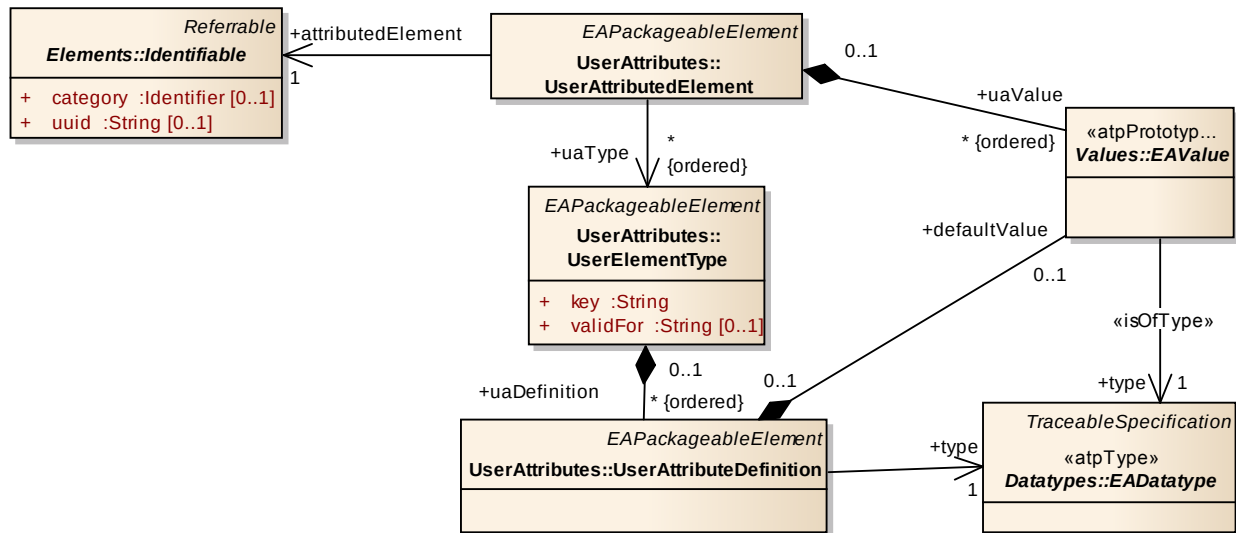


Figure 7. The User Defined Attribute

3.11 Diagram Model

There is no information modelling for diagrams in EAST-ADL. Project Synligare has identified the need for handling graphical model as a way to communicate complex system models in an understandable way. When we use different tools we need a common format to exchange graphical information. After review of a few diagram XML formats, GraphML has been selected for the purpose of study and evaluation of the concepts that pertain to information modelling and communication of models including diagram information. GraphML is a well-documented format that is not tool-specific. It has a rich language set, but we will only use a limited set

There are some needs that the format does not give support for directly:

- Need to support EAST-ADL references (both UUID or name ref)
- The tool/exchange needs to support the diagram abstraction. This means that you can exchange multiple diagrams.
- Ports and sub-elements need to be grouped with its parent element

The limited GraphML diagram exchange format is shown in Figure 9. The exchange of this information is done after the EAST-ADL model exchange, but in sequence.

It is possible to import views without underlying model, but it is not recommended.

In Figure 8, the implementation of the limited GraphML format is shown.

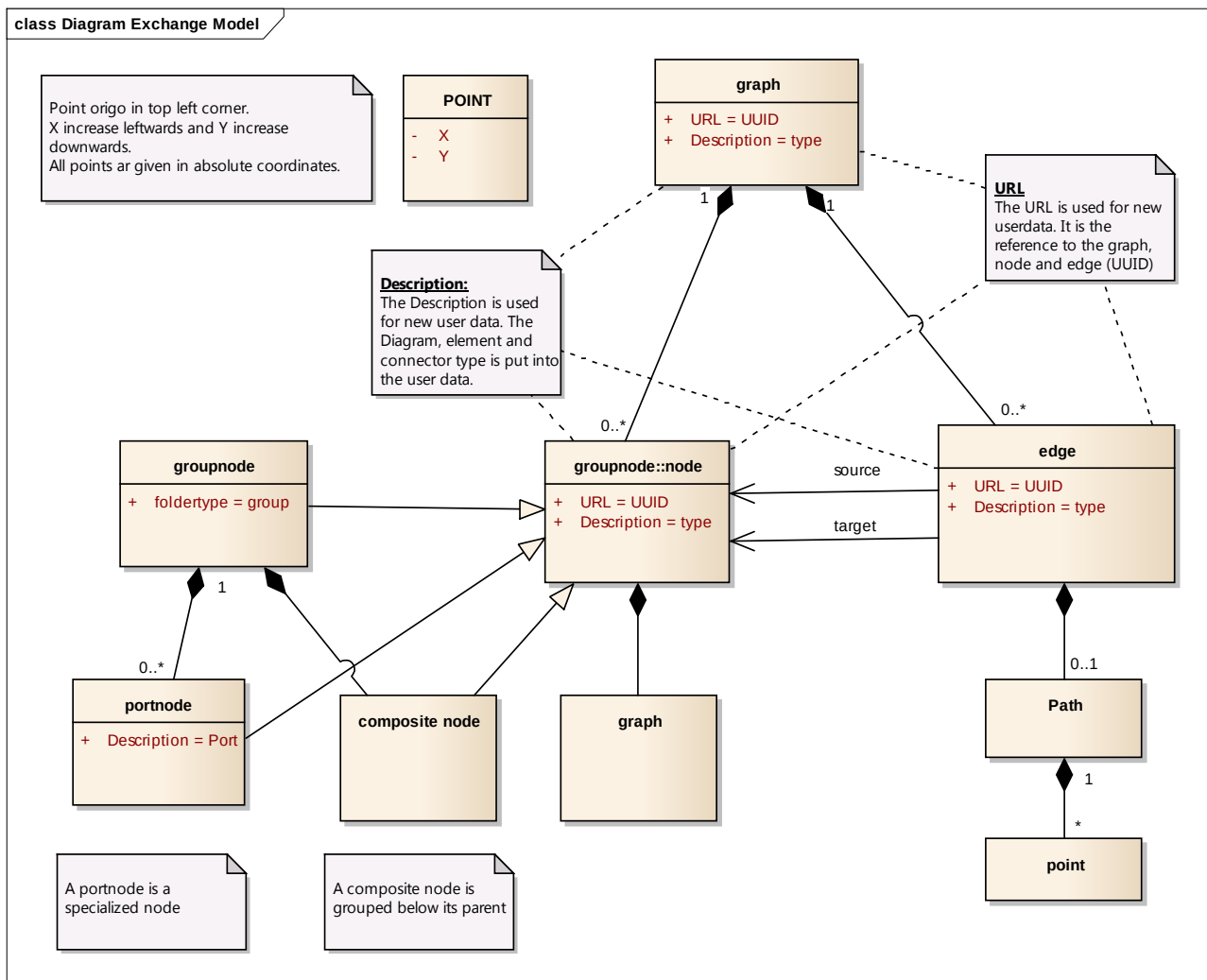


Figure 8 Used GraphML Elements

Figure 9 and Figure 10 show the same diagram in two software tools, namely Enterprise Architect and yEd. Both are capable of displaying GraphML diagrams.

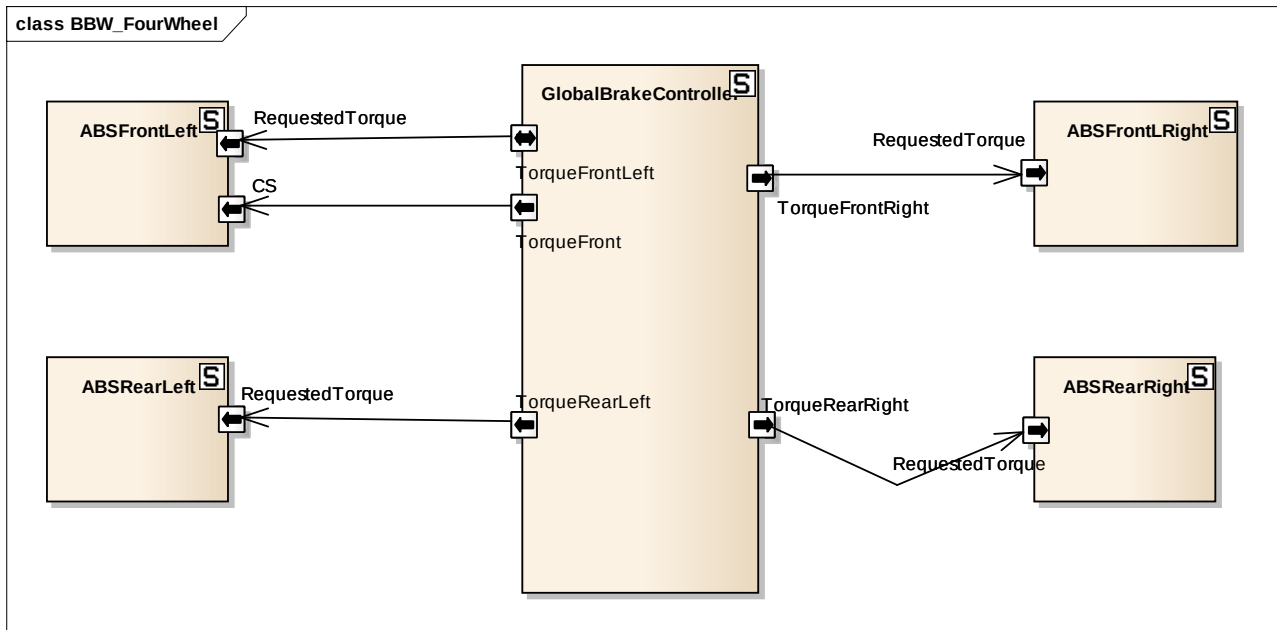


Figure 9 Diagram created in Enterprise Architect

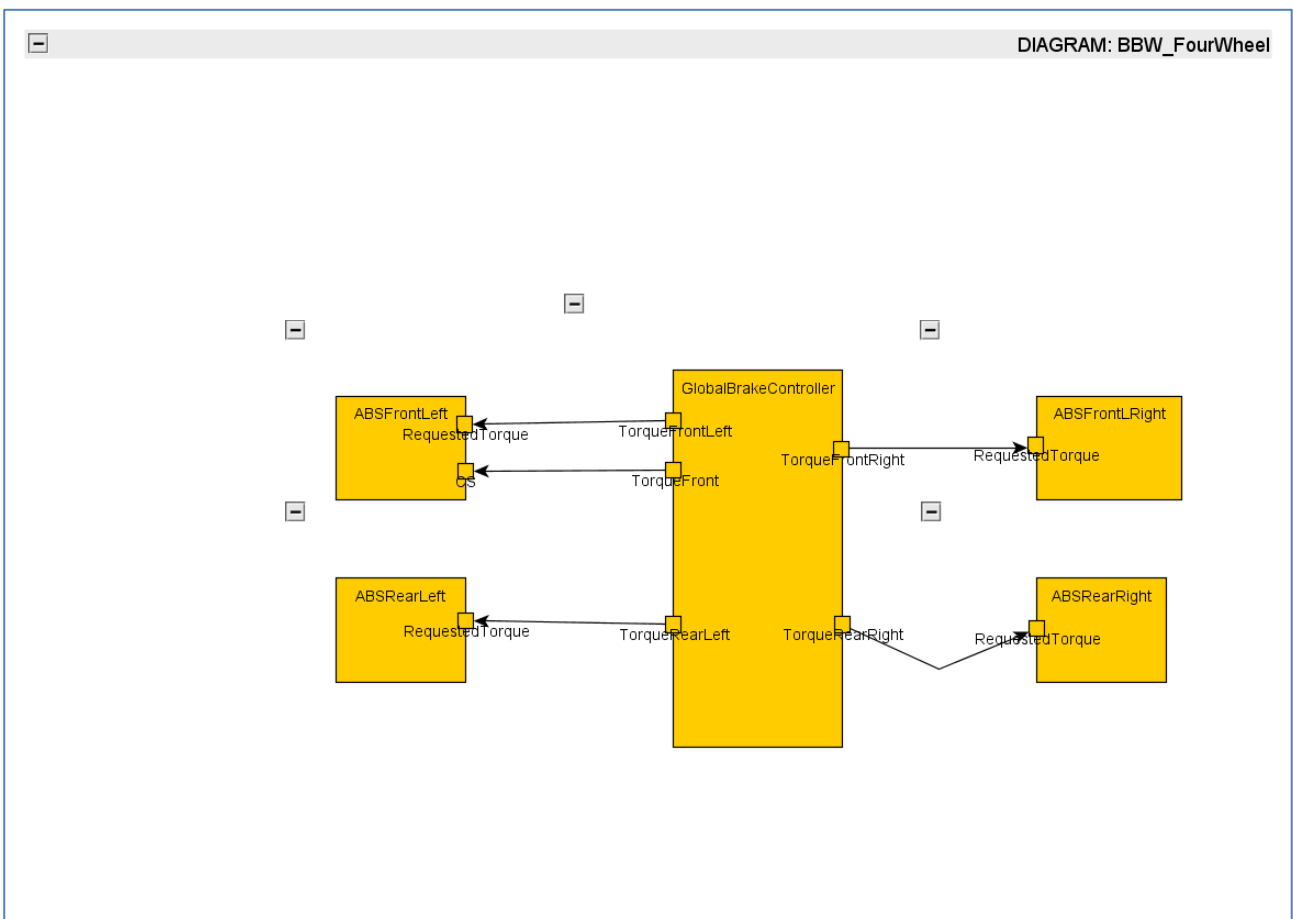


Figure 10 Imported to yEd

3.12 Requirements on Tools for Exchange of Diagrams

Each exchange tool must be able to handle the following:

- Import of a new diagram.
- Previously imported diagrams shall be synchronized during import. It shall be possible to do this in one of these methods:
 - o Either each change can be monitored by the user and be individually accepted/rejected
 - o Alternatively all updates can be shown and the user can do the total import or stop it.

Other demands:

- If the diagrams are imported to a tool that also contains EAST-ADL models the UUID must point to an EAST-ADL element (for diagram elements) or EAST-ADL reference (for diagram connectors).
- The diagram elements and the diagram connectors can be modified in the importing tool. For example more details like colouring etc. can be added. This information is stored within the tool and may optionally be a part of the exchange. If elements and connectors are moved or resized, this shall be exported, as it is mandatory.
- The unique UUID which uniquely identify objects in the core tool shall be stored together with the other data and must be possible to export when the diagrams are exported again.
- An export from an imported diagram file shall produce a semantically identical .xml-file. To make comparison of two diagram files easier, it is beneficial if the order of elements are preserved to the extent that it is possible.
- It shall be possible to differentiate between the old diagrams and the updated but graphical merging is optional.
- A diagram object that no longer is a part of the new (imported) model is not removed during synchronization. A warning is printed and it is up to the user to remove manually.

Even with the requirements that are listed above, any for diagram exchange will have some limitations:

- No support for variability
- No support for versions
- Import/Synchronization is done manually. This means that the imported model may not match if synchronization is done seldom.
- Import is prohibited if two objects have the same UUID.
- Handling of hanging links (links that not yet are solved) is difficult to solve in Enterprise Architect. If diagram object refers to element or relations that are not yet loaded a dummy element must be created because in Enterprise architect a graphical element must refer to a previously defined object. How the references are stored for graphical elements is not yet solved.

4 Manufacturer-Supplier Interaction

This chapter describes processes and process steps appropriate for the interaction between an OEM and a Tier-1. The scenario considered is that an OEM is developing a system and buys a key subsystem from a Tier-1. The OEM has a system model with an interface to the subsystem and a requirement specification for the subsystem. It may be that the Tier-1 is aiming to develop the subsystem as a new development project. An interesting alternative situation would be that the Tier-1 has a model of an existing generic subsystem, such that some development is necessary to modify the existing subsystem to meet the requirements of the OEM. The RFQ-process is conducted to agree on a collaborative development, with its associated cost and time-line. In this section, the following RFQ-related topics will be presented: A traditional RFQ process, a suggested RFQ-process and the exchanges of information that takes place, as well as the associated activities. The purpose is to describe activities that can be supported with views and metrics.

4.1 Traditional Interaction between OEM and Tier-1

Referring to a typical collaborative development project based on requirement specification as traditional, this section considers the interactions between OEM and Tier-1.

The traditional interactions between OEM and Tier-1 in a collaborative development project are shown in Figure 11. The OEM (marked as customer in the figure) and the Tier-1 (marked as supplier in the figure) play different roles. The OEM owns the system requirements and the system integration. The Tier-1 delivers SW, HW or both with increasing maturity as the development project progresses.

A first sub-process, called RFQ-process, starts from the first contact between the OEM and the Tier-1 and ends when the OEM acknowledges and accepts a quotation prepared by the Tier-1. In Figure 11, the RFQ-process corresponds to the steps between “start” (at both parties) and “supplier selection process” (for the customer) and “quotation sent” (for the supplier).

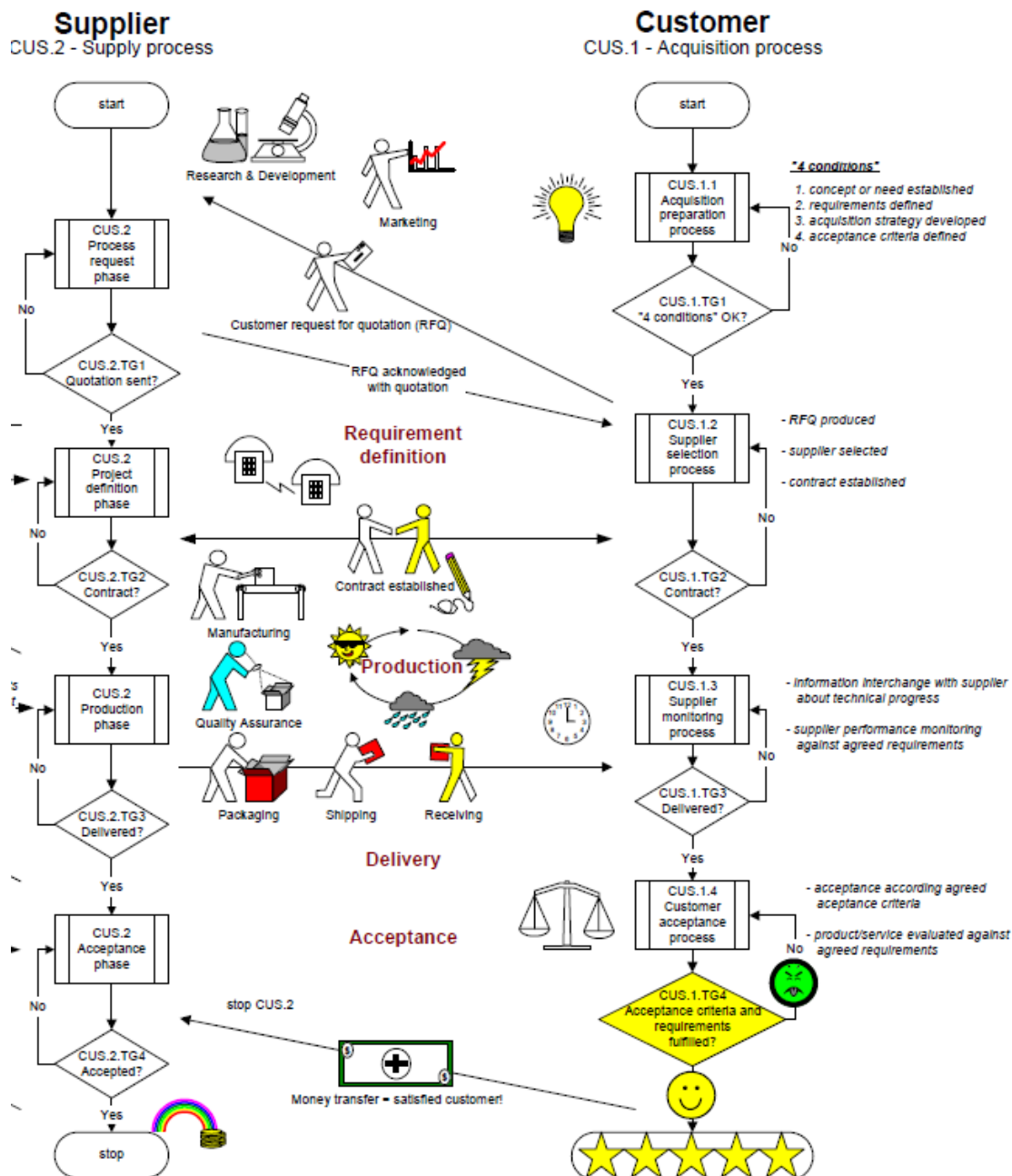


Figure 11 Customer / Supplier process

The interactions and activities between OEM and Tier-1 during the RFQ-process are listed in Table 2. For each interaction, activities at the OEM, activities at Tier-1 and identified views are listed in their respective columns.

Key to the RFQ process is the requirement analysis that is performed by the Tier-1. This analysis involves checking if the requirements can be fulfilled and negotiate with the OEM over requirements that cannot be fulfilled. If there is an existing subsystem that could potentially be adapted to the new requirements, the analysis is performed by attempting to map the requirements to the subsystem components. To correctly estimate cost of any changes that are required, the changes are categorized according to if they involve software, hardware or mechanical development. Certain types of requirements are to be given extra consideration, since they have a more significant impact. Such types of requirements include functional safety requirements, legal requirements, requirements about diagnostic functions and requirements about performance and timing. If material or tooling needs to change, that is taken into account as a key cost factor. A part of the estimated cost is attributed to verification.

In this scenario, it is typical that the OEM has worked out the design of the subsystem to a certain level of abstraction. For example, the OEM might have described the subsystem's functions and have elicited all functional safety requirements. It is then up to the Tier-1 to implement the functions and meet the functional safety requirements.

The responsibility for each requirement should be determined, so that it is known which role in the project is to verify the fulfillment of each requirement, or if indeed the OEM will overtake the responsibility for verifying certain requirements.

Table 2

Interaction	OEM activity	Tier-1 activity	View	Metric
Request for information	Ask for availability of a subsystem or capability to develop that subsystem, by communicating vehicle-level requirements and high-level characteristics	Answer by describing available subsystem or capability to develop the subsystem.	Item definition, Features and requirements	
Request for quotation	Ask for price and duration of development effort, by communicating specific requirements	Preliminary analysis of the requirements, possibly with regard to the existing subsystem, and planning of the development effort, to estimate cost and duration of development effort.	Requirements and features, Requirements and functions, Requirements and system components, Requirements vs architecture	Requirement allocation progress, Realization progress

The next sub-process is the requirement definition process. It continues until an agreement is found and a contract has been signed between the supplier and the customer, as is illustrated in Figure 11. During the requirement definition process the OEM and the Tier-1 can:

- Accept requirement
- Reject requirement
- Change requirement

After the contract agreement it is more complex to make changes in the agreed requirements. But in almost all projects this is necessary.

The requirement analysis in this requirement definition process is detailed in contrast to the analysis performed in the RFQ-process. Here the Tier-1 will explicitly map the requirements from the OEM to subsystem components. A plan is made for what changes are made to the generic subsystem. All changes as well as additional development on top of the generic subsystem are defined as a delta, and marked as such in the plan.

Each function is analyzed and an appropriate way to describe the outcome of the analysis is a block diagram with information flow, where events that trigger functional behavior are marked.

Further information to record in the function analysis include I/O mapping, signal specifications for interfaces and specification of any algorithms, as well as supporting functions, such as the operating system, the diagnostic functions and the I/O system.

The next step is component design analysis, the purpose of which is to get a detailed understanding of the delta development. In this analysis, HW is analyzed in terms of function, environmental conditions and mechanical aspects, as well as sensor (input) and actuator (output)

devices. SW is analyzed in terms of function, timing and performance. The following aspects are specified: I/O interfaces, diagnostic functions, maintenance functions, operating system, communication protocols. The application software is further described, to take into account the changes between a generic algorithm and the new algorithm as required in the delta development. A plan is made for infrastructure that is required for the development work, such as simulation capability.

A test plan is created to ensure that important aspects of the changes are to be properly tested.

During this sub-process, the OEM as the customer go through a supplier selection process, as can be seen in Figure 11.

Table 3 lists activities at the OEM and at the Tier-1, with their respective views and metrics.

Table 3

Interaction	OEM activity	Tier-1 activity	View	Metric
Accepting, rejecting and changing requirements	Answer the Tier-1's requests to reject or change requirements, by analyzing the impact of such changes	Analyze the requirements in detail and if necessary, reject or recommend a change of requirements.	Features and requirements, Functions and requirements, Hardware Components and requirements, Requirements vs. Architecture, Functional and Technical Safety Concept	Requirements allocation progress, Realization progress, Safety-related progress metrics
Supplier selection	Compare the quotations from different suppliers. If they have existing subsystems, they might differ in terms of performance and other quality attributes, as well as the cost and duration of development effort.			Product metrics (supplied by the Tier-1)

After the contract has been signed, the next sub-process is for development and production. It contains more than is illustrated in Figure 11. In this sub-process, there may be early integration and testing performed on early prototypes and for this purpose it is necessary to exchange detailed interface specifications, as well as progress reports to synchronize the time plans of both parts. As development progresses, reasons to adjust the requirements may be found. As design verification progresses, such status may be communicated. If the subsystem includes hardware, the quality of components may be reviewed. Further, functional safety-related artefacts may be communicated and design decisions may be discussed and agreed on. Table 4 lists activities at the OEM and at the Tier-1, with their respective views and metrics.

Table 4

Interaction	OEM activity	Tier-1 activity	View	Metric
Exchanging interface specification	Integrating early prototypes	Selecting hardware components and verifying that the interface fulfills specifications	Hardware components and requirements	
Exchanging progress reports		Reporting the status of design, development and early verification	Illustrating metrics	Realization progress, Verification progress
		Development and integration of subsystem components	Comparing multiple model objects, Software components, ports and signals, Hardware components and requirements	
Integrating and prototyping	Integrating prototype subsystem	Sharing integration information, release notes, verification reports, etc	Illustrating metrics Hardware components and requirements Functions and requirements	Product metrics Verification progress
		Functional safety analysis	Error propagation modelling	
Changing requirements		Discovering problems in meeting the requirements while selecting hardware components and while developing	Features and requirements, Functions and requirements, Hardware components and requirements, Requirements vs architecture	
Exchanging Functional Safety-related information	Requiring functional safety documentation for the purpose of in-vehicle trials of prototypes	Reporting functional safety documentation, as well as progress on implementation of diagnostic functions	Illustrating metrics, Functional and Technical Safety Concept	Safety-related progress metrics

Next and final sub-process of interaction between OEM and Tier-1 is delivery and acceptance, as can be seen in Figure 11. At this point, the status of all artefacts should be complete and documentation should be mature.

Table 5

Interaction	OEM activity	Tier-1 activity	View	Metric
Integration	Integrating the final version of the subsystem into the system	Supporting integration with relevant information	Illustrating metrics, Hardware components and requirements, Functions and requirements	Product metrics
Reporting final status		Delivering reports of completed artefacts	Illustrating metrics	Realization progress, Verification progress, Requirement validation progress
Delivering functional safety documentation		Delivering functional safety documentation	Illustrating metrics, Item definition, Functional and Technical Safety Concept Error propagation modelling	Safety-related progress metrics
Reporting documentation		Delivering various bits of documentation	Various views	Various metrics

4.2 Suggested Interaction between OEM and Tier-1

A key difference when comparing traditional interaction between OEM and Tier-1 with the interactions that are made possible by EAST-ADL is the model based design paradigm. EAST-ADL enables exchange of models. Instead of the traditional communication of textual requirements, it is plausible to envision exchange of normative and informative models complemented with textual requirements. For this purpose, it is not enough to communicate on a general modelling language. To make normative models effective in communicating clearly, a clear and industry-specific semantic is required. In this regard, EAST-ADL is more appropriate as information model than SysML and UML.

The introduction of a model based approach will impact this process in a number of ways, which both have pros and cons. The key benefit of introducing exchange of models in place of requirements is the transfer from a textual format to something more visual, which is expected to give overview. In the traditional exchange of requirements, the number of requirements could be excessive and this would make it hard to get an overview. In fact, it is likely that much of the information that was traditionally communicated as requirements is better represented in models. By giving more overview and reducing the number of requirements, it should be easier to communicate, analyze and review requirements.

When modelling is made on a deeper level than traditional, it is important to choose the right level of details. A lot of requirements could be moved to design, but they must still be implemented and verified.

A model-based development process that benefits from common information modelling is presented in Table 6. It should be noted that Table 6 is not as detailed as the tables in the previous section.

Table 6

OEM activity	Tier-1 activity	View	Metric
Defines the intended functionality of the vehicle		Feature tree	
Defines requirements and use cases for the features		Features and requirements	
Identifies parts of the functionality to buy from suppliers			
Defines system model and keeps parts to be bought as black boxes		Feature tree Functions and requirements Functional and Technical Safety Concept Hardware components and requirements Software components, parts and signals	Requirements allocation progress Safety-related progress metrics Product metrics
Sends system model to supplier as a request for information. The content and level of detail corresponds to the OEM requirements.	Analyses system model	Features and requirements Functions and requirements Comparing multiple model objects	Requirements allocation progress
Analyses refined system model	Sends back refined system model. The content and level of detail is restricted to what is available up front and what can be shared from a secrecy perspective.	Features and requirements Functions and requirements Comparing multiple model objects	
Sends system model to supplier as a request for quotation	Analyses system model	Features and requirements Functions and requirements Hardware components and requirements Requirements vs	Requirements allocation progress Requirement validation progress Product metrics

		architecture Comparing multiple model objects	
Analyses refined system model and quote. The OEM agrees on the quote.	Sends back refined system model and quote	Features and requirements Functions and requirements Hardware components and requirements Requirements vs architecture Comparing multiple model objects	

The steps of sending system models and analyzing system models in **Error! Reference source not found.** can be iterated until there is a consensus. The minimal content is a Feature Model defining the system content, but typically there will also be interface specifications to secure that the negotiated system can be integrated.

The same interactions as in the traditional interaction between OEM and Tier-1 can be expected, but based on models with complementary requirements. In the model based development context, we expect that it is possible to semi-automatically gather and export progress and product metrics, as well as some views. In other words, the manual effort of keeping up the communication between OEM and Tier-1 is significantly reduced. The hope is that Project Synligare will demonstrate that this reduction in effort is indeed achieved.

5 Metrics

Various metrics on engineering documentation can be used to assess the product's properties or the development progress. In Project Synligare, we are investigating metrics that can assist the collaborative aspects of development, with particular focus on safety aspects. In general, all metrics that assist planning and follow-up engineering work are beneficial from a safety perspective, since faults caused by lack of resources and lack of time are impeded.

A metric is calculated on the basis of an entire system specification or parts thereof. Examples of delimitations include

- Architecture subset - The metric concerns a particular architecture or parts thereof
- Requirement subset- The metric concerns a particular requirement set
- Project - The metric concerns a particular project

A hierarchy of metrics based on the structural hierarchy of the system (system, subsystem, etc.) is conceivable. Depending on character, metrics can in a tool or in a report be visually represented as lists, flags and colors in addition to plain numbers.

Two kinds of metrics will be discussed below, progress and product metrics. Development progress can be assessed based on the status of the model or based on assessments of actual status that is reflected in the model.

5.1 Requirement Validation Progress

In order to use requirements to communicate product prerequisites and specifications and have an appropriate input for detailing the product, they have to be validated. As this is also a measure of progress, a Requirement Validation progress metric can be defined.

A requirement is considered valid when its content has been assessed. This may be marked using an attribute, or the requirement can be moved from a preliminary to its final package after validation. For the purpose of this metric, we will use the latter method: Initial requirements are placed in a preliminary package RequirementModel, and as they are validated they are moved to the correct package or RequirementModel. In order for the metric to be calculated, the package for preliminary requirements is marked with a "Preliminary" attribute using user defined attributes. An enumeration datatype with one value "Preliminary" is needed for this purpose. Figure 12 shows the relevant metamodel elements for attributing the preliminary package or requirement model (top right) and the requirement, package and requirement model for organizing the requirements (bottom).

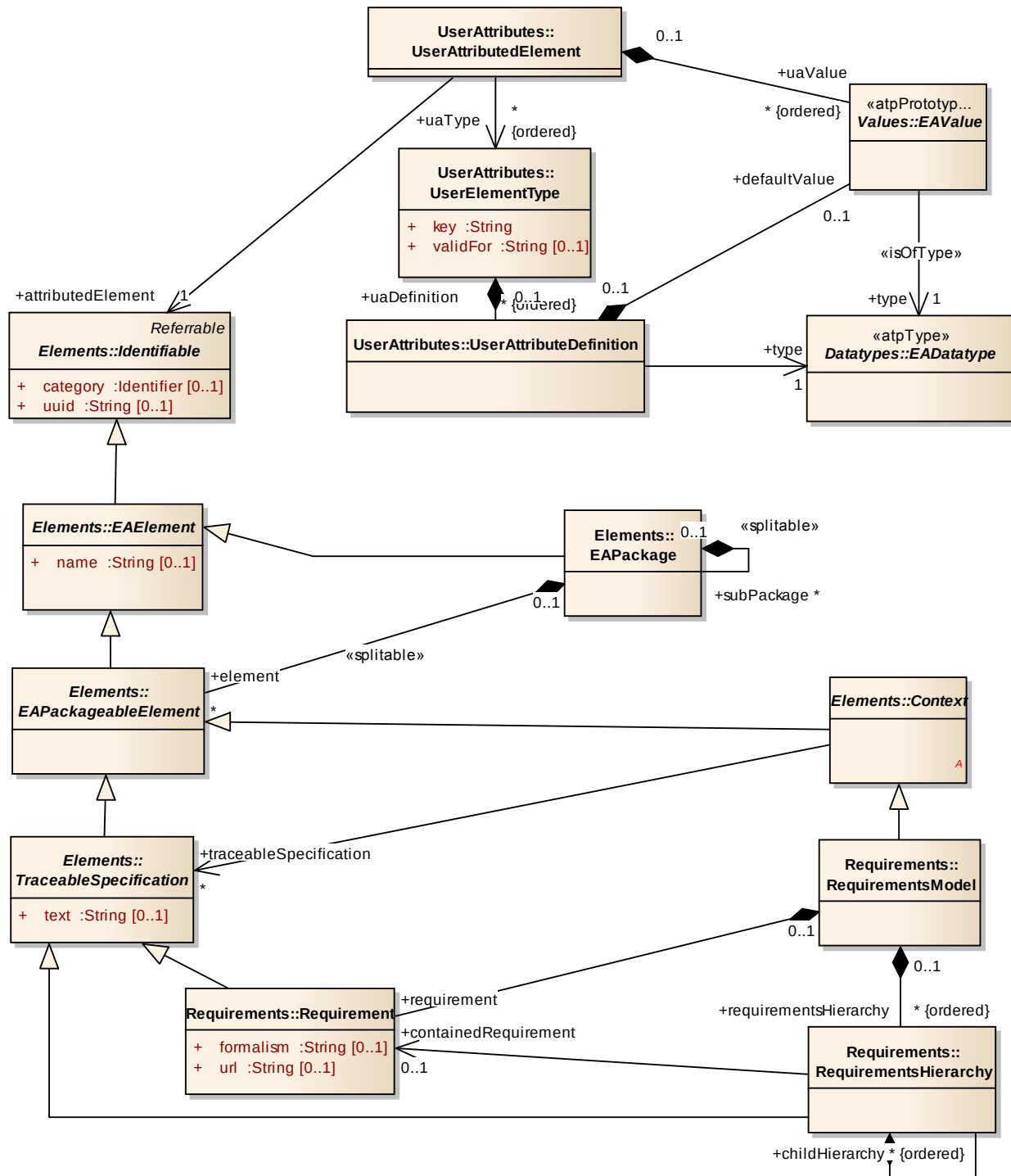


Figure 12  Non-validated requirements can be located in a package or requirements model marked with a user defined attribute with value “Preliminary”

5.2 Requirement Allocation Progress

Assuming that requirements are used to specify what the product shall do, they can also be the basis for assessing progress. Progress of requirement allocation is measured as the fraction of requirements allocated to architectural elements.

The metric for progress of requirement allocation provides an example of a metric that counts the number of completed tasks (see the scope defined in Section 0) with respect to covering the whole system. Typically, this type of metrics is used to follow the progress of a development project.

Figure 13 shows modelling elements for the satisfy relationship to illustrate that the requirement allocation progress metric is such that a requirement can be considered allocated when a) there is a satisfy link or b) when its derived requirements have satisfy links.

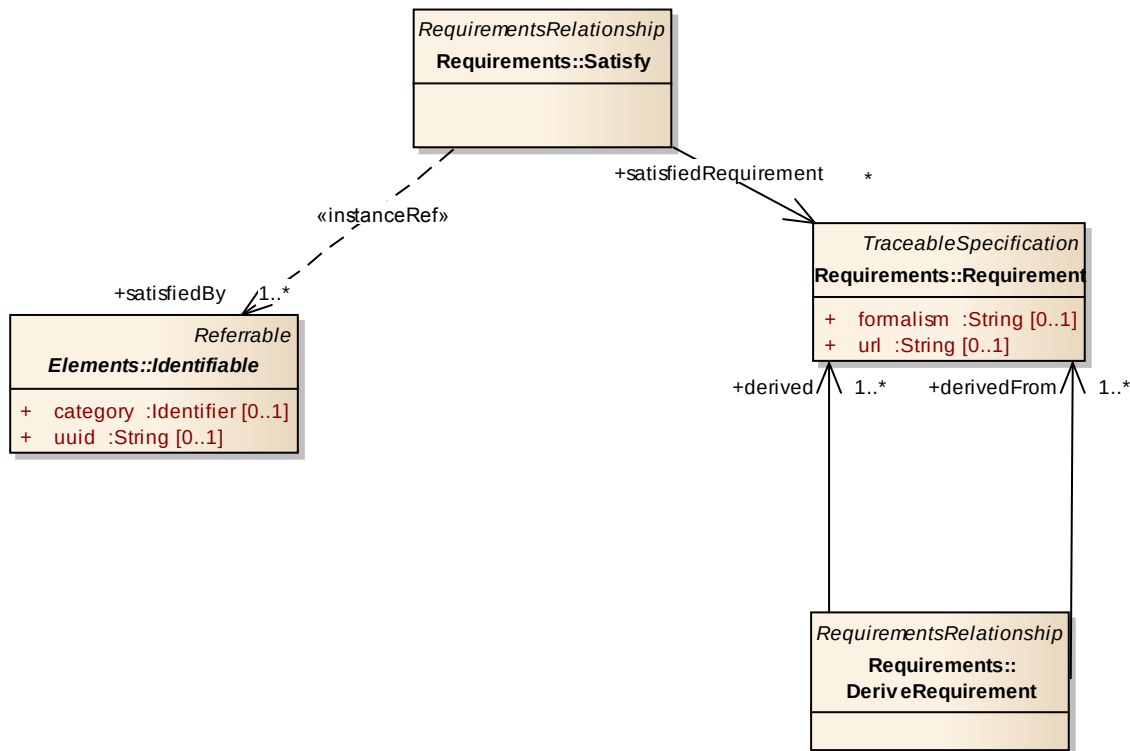


Figure 13 Requirements can be considered allocated when they have satisfy links, or their derived requirements have satisfy links.

Because requirements may be derived to more specific requirements, a requirement with one or more derived requirements is also considered as indirectly allocated when the derived requirements are allocated. Such “base” requirements are thus not counted among the allocateable requirements. The exception is when derived requirements are located in another package or requirements model. In this case, the requirements are considered to be at another abstraction level and therefore they do not qualify the “base” requirement as allocated.

Progress of requirement implementation is measured as the fraction of requirements that are implemented in the product. This criterion is not well-defined, as it may concern requirements with a final design, final software specification, final code, compiled code, etc. What we will assume is that a requirement that has derived requirements that are allocated to Implementation level elements is implemented.

On computing this metric, the model can be inspected for one of the criteria mentioned, or a specific flag that is manually set by the engineer who is performing the task of allocating requirements is used as a basis.

5.3 Realization Progress

The EAST-ADL model [3] provides means to relate model elements to its abstract counterpart. For example, a function or ECU is linked to the Vehicle Feature it is realizing. In the end all Vehicle Features should be realized down to the Software Component level. On computing realization progress, one must decide which criterion to use, for example realization in software, function or hardware architecture. One must also be aware that it is not possible to know how many concrete elements are required for realizing an abstract element. For this reason, an alternative approach is to use a specific flag which is manually set when a feature is deemed to be realized.

This metric of realization progress in an example of a metric used to count the number of completed tasks (see the scope defined in Section 0), in this case the task of modelling on a lower abstraction level and adding realization links between the elements of the more abstract model and the corresponding elements of the more specific model.

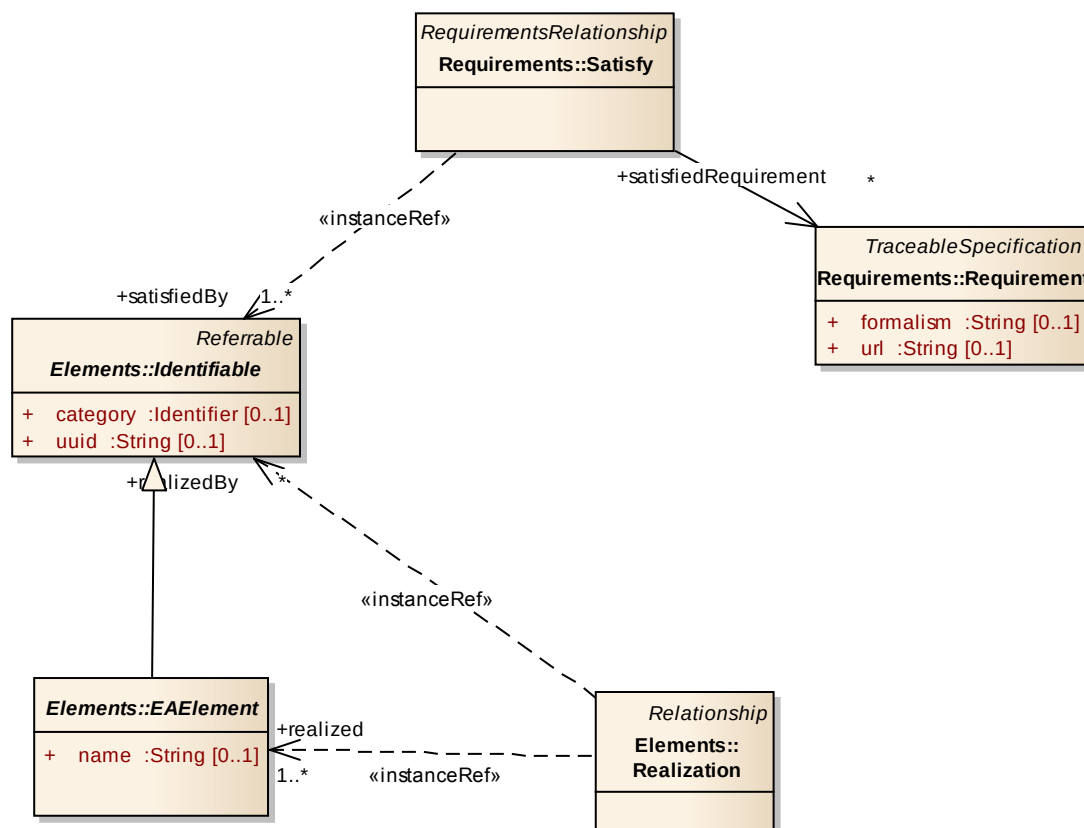


Figure 14 Requirements can be considered done when they are implemented, i. e. their target architectural element is realized.

Figure 14 above shows the realization relation between artifacts, and Figure 15 shows how to attach flags using the user attribute mechanism.

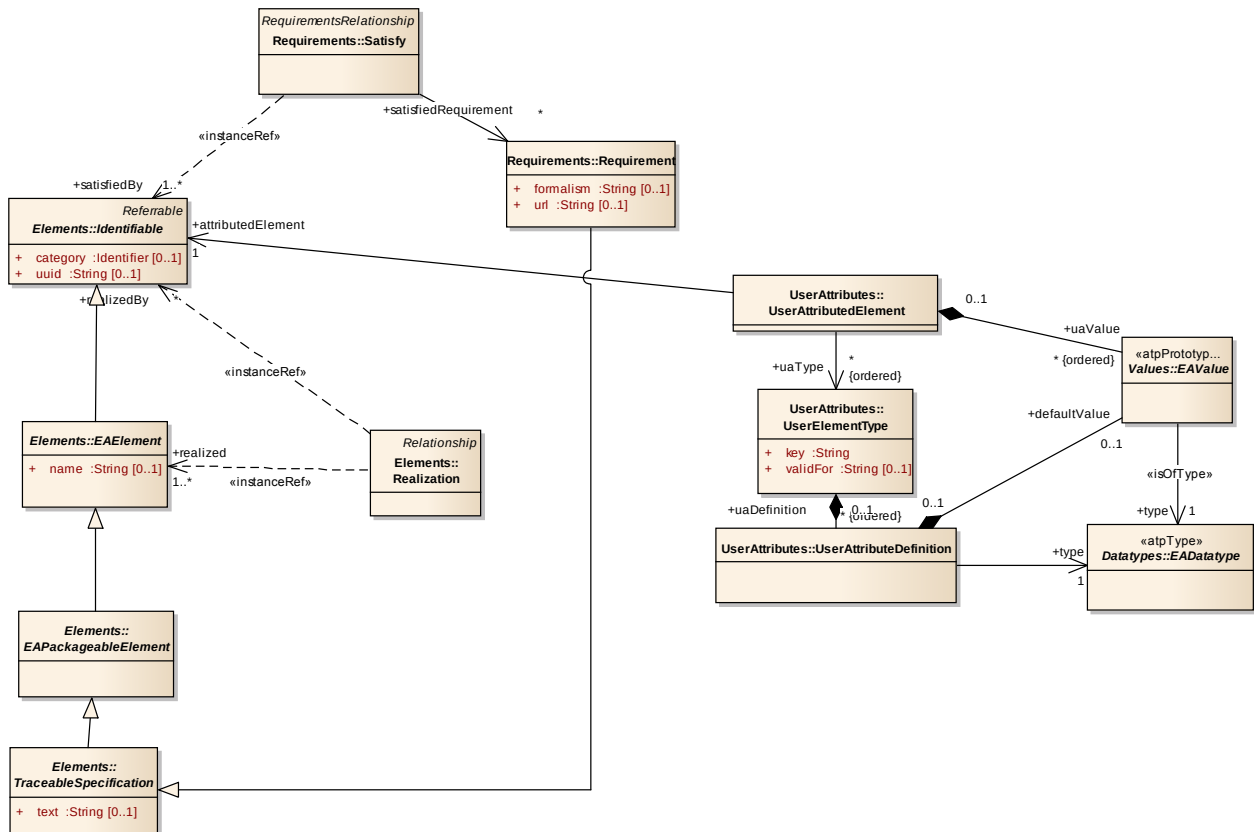


Figure 15.  User Attributes can be used to put flags on model elements

5.4 Verification Progress

In EAST-ADL, the verification specification VVCase is linked to requirements. For each requirement one or several verification cases can be defined, each of which may pass or fail. The verification progress metric counts the number of performed verification cases with respect to the set of verification cases that are defined in a verification plan.

This metric for verification progress contributes to the scope for this report as defined in Section 0.

Figure 16 shows how requirements and verification are related. Note that there is an abstract VVCase defining the test or analysis steps required, and an abstract VVCase that holds one or several verification results.

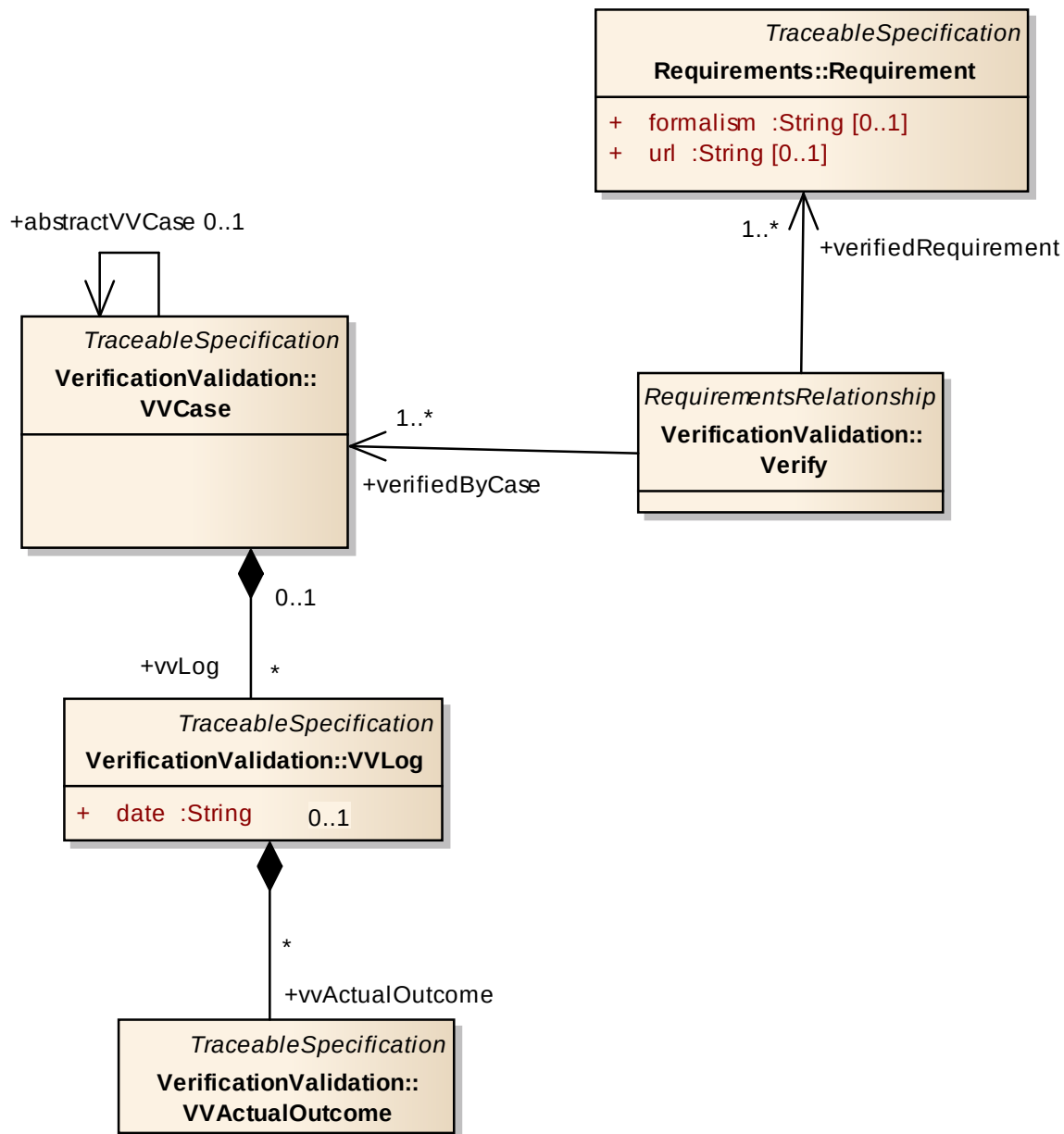


Figure 16. Verification specification for Requirements

Verification progress may thus concern the fraction of requirements that are verified or where a verification method is defined. If multiple verification cases are defined per requirement, the fraction may be based on verification cases instead. In case verification is done, a useful metric is the fraction of passed vs. failed requirements.

5.5 Safety-related Progress Metrics

The ISO26262 functional safety standard [4] defines a number of engineering artifacts that shall be defined in the context of its Item. One of these artifacts is the safety goal, which is the starting point for organizing safety requirements. Examples of metrics include the fraction of Items with safety goals defined or the fraction of safety goals with functional safety requirements. Because it is system specific, it is not possible to know how many safety goals or safety requirements are needed. It is still a useful metric, since the amount must be non-zero.

These safety-relevant progress metrics are examples of metrics that count the number of completed tasks, as mentioned in the definition of this report's scope in Section 0.

Another safety-related metric is the fraction of functional safety concepts that are matched by a technical safety concept. This can be computed based on derivation relations between functional and technical safety requirements combined with realization relations between analysis functions and design functions making up the architectural aspects of the functional and technical safety concepts, respectively.

Figure 17 shows how the FunctionalSafetyConcept element identifies a set of Requirements. These requirements together with the elements to which they are related with a satisfy relation makes up the FunctionalSafetyConcept. The corresponding SafetyGoal is found through the Derive relation to the requirement identified as a safetyGoal.

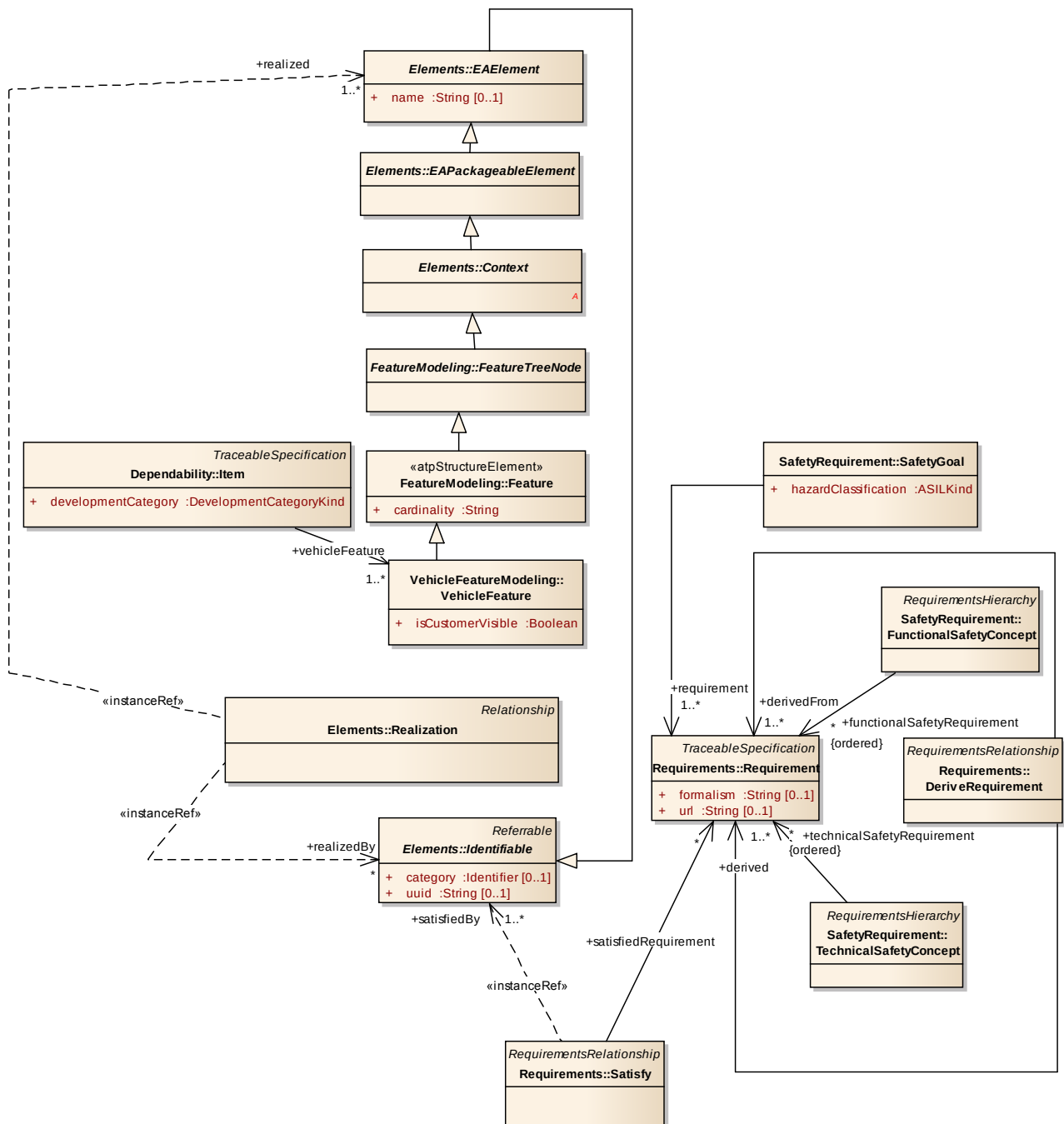


Figure 17. Elements and relations related to Functional and Technical Safety Concept

5.6 Product Metrics

As opposed to progress metrics, product related metrics are typically absolute. One metric that is relevant for strategic decisions regarding development rigor or priority, is the number of manufactured vehicles that are impacted by a certain engineering artifact. This can be computed based on the take rate annotations of vehicle features realized by the engineering artifact considered. Similarly, total revenue or cost can be computed by combining take rate information with financial annotations.

When applied to estimate production volume and cost factors, the product metrics contribute to the scope defined for this report in Section 0.

With product metrics, it is possible to track dependability expressed in the expected lifetime of hardware components with respect to the expected stress that they will be exposed to. This is typically calculated in a structured way. It can be seen that such a metric contributes to the scope for this report as defined in Section 0.

Figure 18 shows how GenericConstraints of kind pieceCost can be used to define cost. The TakeRateConstraint defines with an absolute number how many elements are produced of the target, if the source is empty. If a source is also identified, the take rate is a fraction.

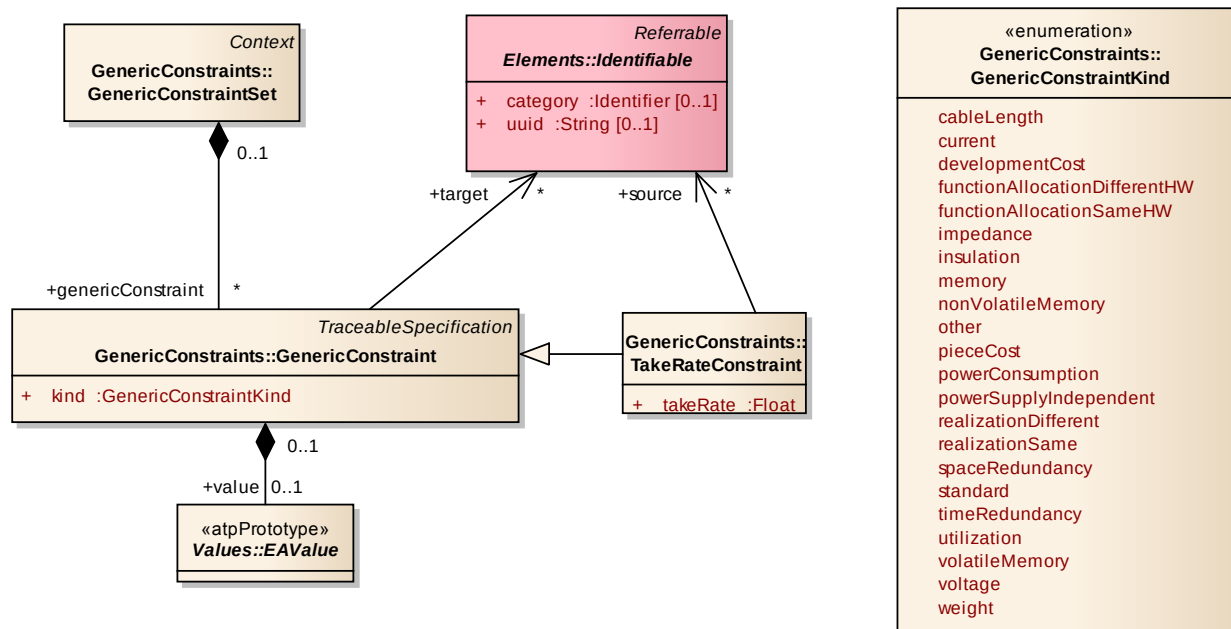


Figure 18. GenericConstraint can be used for annotating elements with a cost and take rate.

Various dependability metrics are relevant for developing safety-critical systems. These include failure rate and other hardware architecture metrics. Figure 19 shows elements relevant for failure rate annotation, based on which it is possible to calculate a hardware dependability metric.

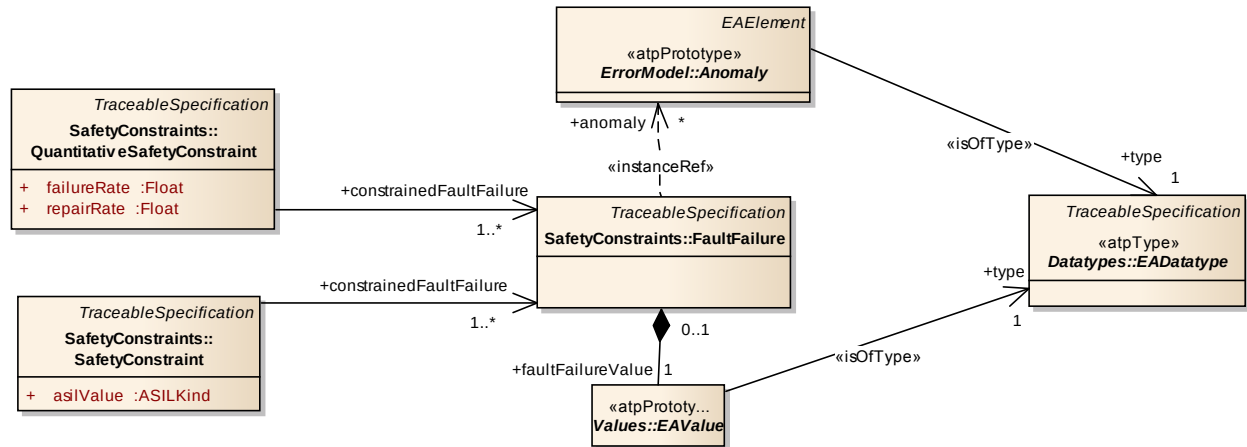


Figure 19. Failure rate constraint as example of product metric.

5.7 Identifying Context

In general, it is necessary to be aware of context when computing metrics based on fractions. The AUTOSAR/EAST-ADL notion of package is one possible context. Another is to consider a certain feature, function, hardware component and compute a fraction among the related elements.

6 Views

A view is a concept where a subset of the available system information is collected. The information that is modelled in a view is dependent on the needs of the parties involved in development. The scope of views considered in this report is given in Section 2.2.1.

In this section, views that have been processed in WP2 are described and illustrated. View examples are marked , while metamodel diagrams showing the related modelling concepts are marked .

6.1 Generic Tree View

Elements in a model may be organized in a containment hierarchy based on packages. Typically, it is necessary to browse such model according to the containment hierarchy, but also along various relations. Examples of important relations are type relations, instance reference relations and relation elements, see [Figure 9](#). In a generic tree view, it is typically relations of the type parent-child that are shown, since this type of relations lend themselves well for a tree view. However, other types of relations also need to be navigable (for example model references) in order for the view to be usable with large models.

The preferred concrete view is categorized as a tree view with a set of features to:

- visualize the model
- navigate the model
- visualize model relationships
- visualize selected model object context
- visualize and categorize model object types
- create model objects

6.1.1 Example View

EATOP holds an implementation of a tree view, see [Figure 20](#). It is currently being augmented according to navigation needs described above.

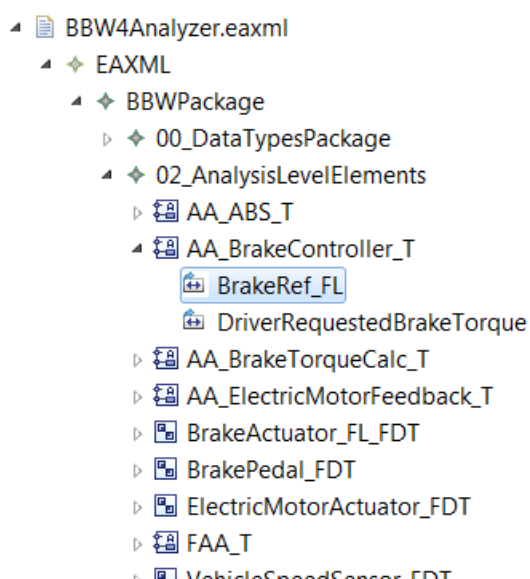


Figure 20  Example of a tree view

6.1.2 Related Modelling Concepts

Figure 21 shows relations that are relevant for navigation between elements in a tree view: Navigating the specializations of Relationship, such as Realization, Satisfy or Verify are important to get a quick overview of a model. The is-of-type relation is fundamental to access the specification of an element. Many relations are of type instance-ref, i.e. they refer to an element in a deep prototype hierarchy (an instance hierarchy of reused function, hardware or software types). Such navigation relies on a combination of the is-of-type and instance-ref relations.

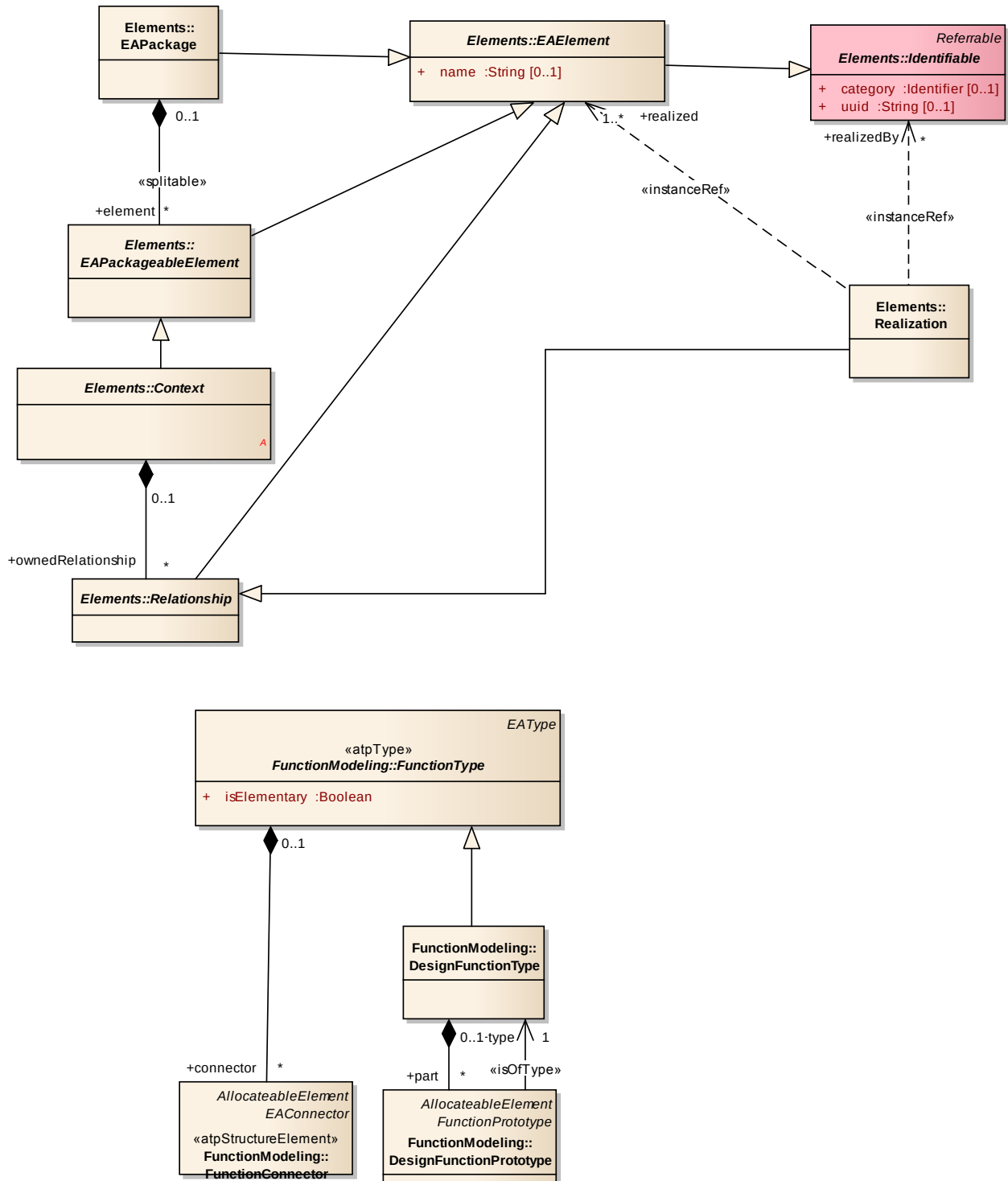
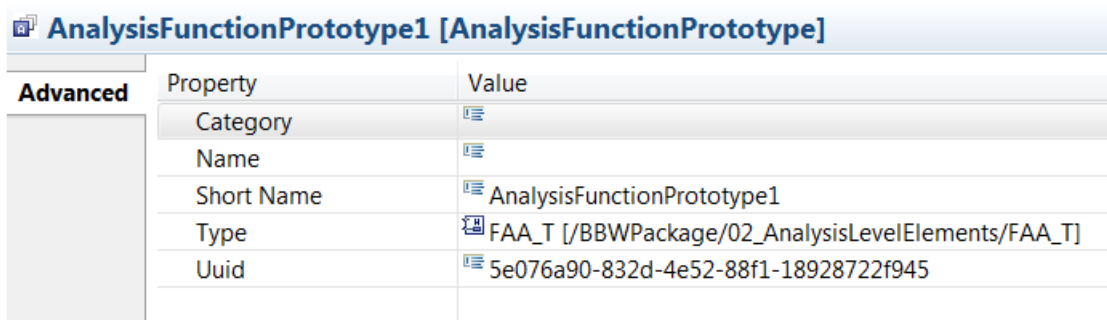


Figure 21 Example of relations that are relevant for visualization and navigation

6.2 Viewing Related Model Objects

Elements in a model usually contain some kind of data about the element and references to or from the element. Viewing such data can be helpful whilst either trying to understand the model or looking for problems and errors. Conceptually, this is done by viewing one object at a time and the data is then presented in a table or list. An example of this can be seen in Figure 22 which depicts an element's own attributes. However, it can be difficult to compare this extract of data between different elements simultaneously. Currently, the way to do this would be to first view one element and then view the other and try to discern the difference between them. Furthermore, the complexity of comparison scales in accordance with how many elements are involved. A concrete example where this type of view is applicable is when showing and editing which functions are allocated to what component of the hardware architecture.



Property	Value
Category	
Name	
Short Name	AnalysisFunctionPrototype1
Type	FAA_T [BBWPackage/02_AnalysisLevelElements/FAA_T]
Uuid	5e076a90-832d-4e52-88f1-18928722f945

Figure 22 Example of a properties view from Eatop's EAST-ADL explorer

Conceptually, it could aid the process if one were to have a view which could compare and edit a large number of elements simultaneously. One way to do this would be to hold the data of the elements in a table. Where the elements name could represent the rows and the columns could represent the data.

This view contributes to the scope of the report (see Section 2.2.1) as a way to illustrate how different elements are related. The preferred concrete view is categorized as a table view.

6.2.1 Example View

Figure 32 shows an example view, where the main elements are generic constraints in rows and attributes or associated elements in columns.

The screenshot shows the EATOP - Eatop Technology Demonstrator interface. On the left is the 'Project Explorer' showing a tree structure of model elements. The main area displays a 'Table View' with the following data:

	value	mode	kind	target	Satisfied Requirements	Incoming references
cableLengthUnit	1		CABLELENGTH	BrakeActuator_AT CPUtempSensor_T FlexRayWire1		
cableLengthUnit2	2		CABLELENGTH	Wheel_FlexECU_T WheelSpeedSensor_AT		
cableLenOverall	100		CABLELENGTH	HDA_T		cableLenRef
PowerConMax	100		POWERCONSUMPTION	HDA_T		powerConRef
PowerConUnit2	2	Mode2	POWERCONSUMPTION	Wheel_FlexECU_T WheelSpeedSensor_AT		
PowerConUnit3	10		POWERCONSUMPTION	CPUtempSensor_T		
PowerConUnitA	3	ModeA	POWERCONSUMPTION	HC1		
PowerConUnitB	7	ModeB	POWERCONSUMPTION	HC2		

Below the table, there are tabs for 'BBWPackage', 'default.eaxml', and 'cableLenOverall'. An 'Overview' button is also visible.

Figure 23 Example of a table view in EATOP with GenericConstraints in rows and properties and associated elements in columns

6.3 Creating connector elements

Using a tree view also involves managing the model and creating additions to the mode, such as creating connectors and ports. Depending on the element type the process of creating a new element may have varying degrees of complexity. Creating a connector is tedious. This task involves the creation of the connector itself, creating the port reference elements and finally linking these to actual ports. Visualizing and automating part of this process simplifies and speeds up the process of working with models. A suggested concept is to create a tool which can parse the model and present options for the user of relevant ports, bringing down the process of creating connectors to a couple of steps.

6.3.1 Example View

Figure 24 shows an example view with ports from the local context that are accessible for selection and automatic connection.

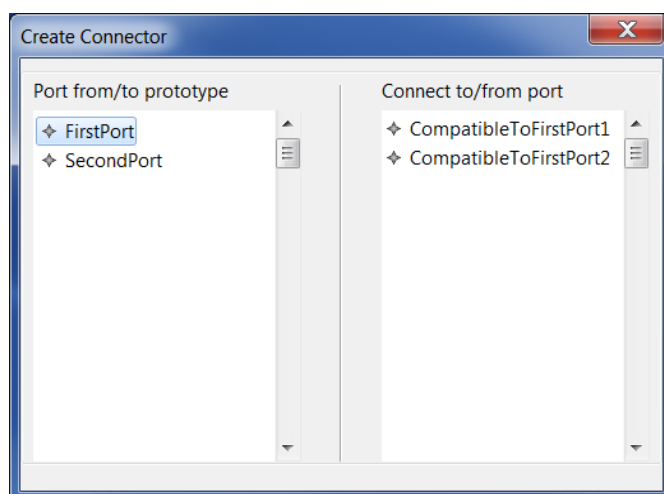


Figure 24 Concept of creating a connector

6.3.2 Related modeling Concepts

Figure 25 shows the port and connector elements, which are the basis for a connector view.

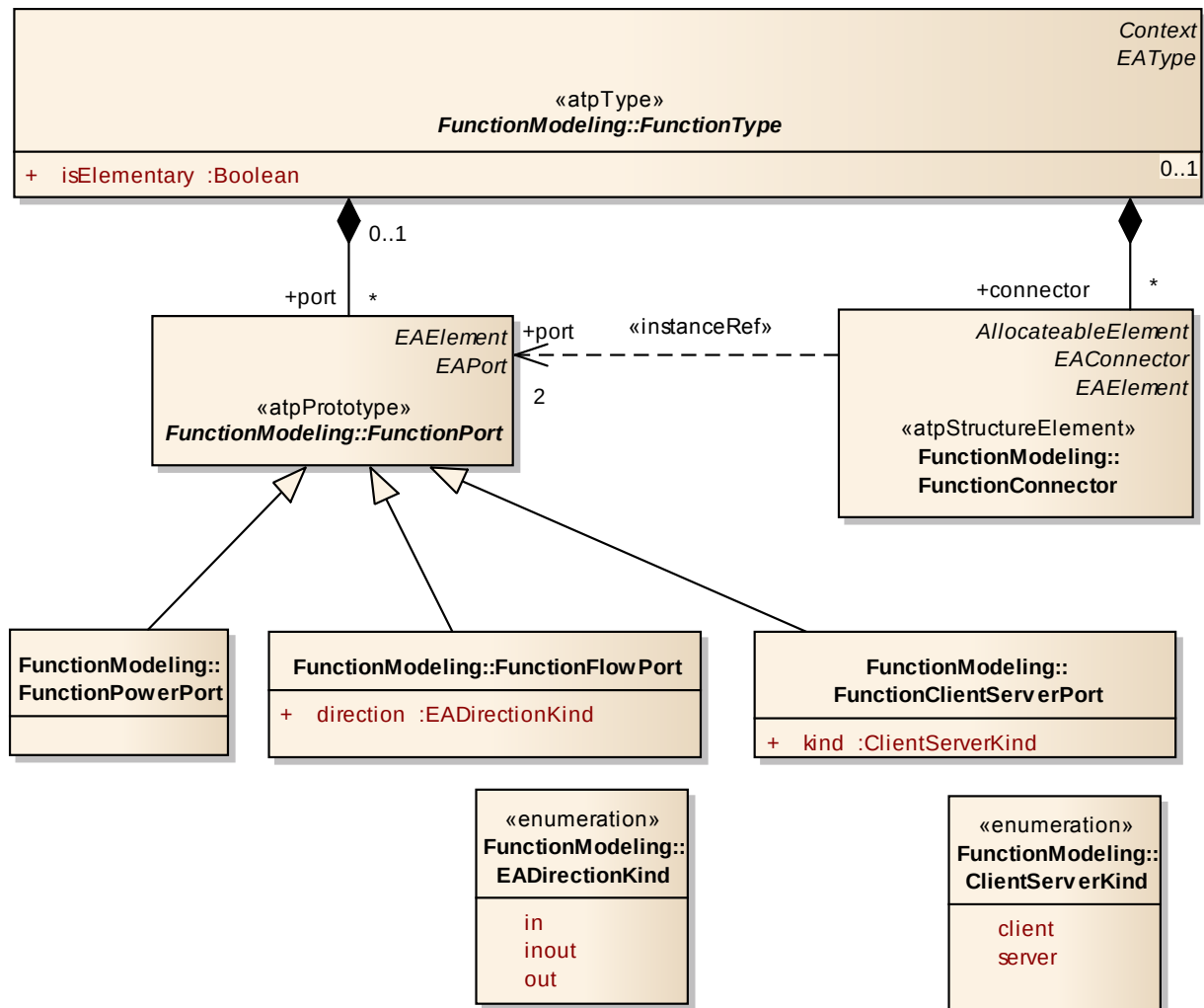


Figure 25. Ports and Connectors

6.4 Compare and Merge

An important use case during system development, in particular in a collaborative setting, is the merging of models of different origins. Models may represent a new version of something that already existed, in which case it is a difficult task to identify changes and decide whether to accept or reject the change. The assessment is easier if version information is included, as the default choice would then be the most recent element.

Compare and merge is a view and editing support that assists the engineer during comparison of versioned or non-versioned changes to models and during updating the base model based on the comparison results.

6.4.1 Example View

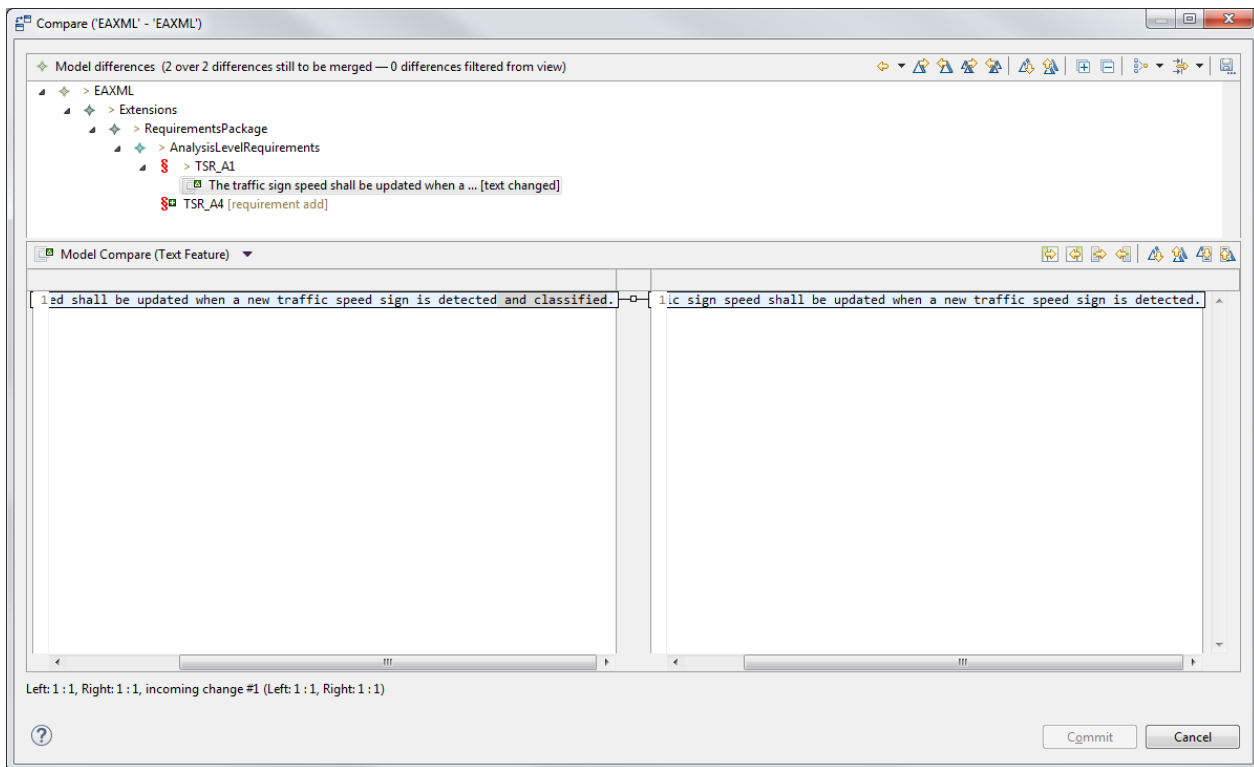


Figure 26 Screenshot of Compare and Merge.

6.5 Increments of Features

On a feature-level modelling of a system, the main variability is handled. A typical way of illustrating a feature model is the feature tree. It has a semantic of mandatory features and optional features, where the optional features are included in the system on an exclusive-or basis.

The view that shows a feature tree contributes to the scope for the report as defined in Section 2.2.1.

The preferred concrete view is categorized as a tree view.

6.5.1 Example View

Figure 27 illustrates a feature tree decorated with steps of incremental development in the form of numbers in the top left corner of each feature symbol and functional safety information in the form of ASIL. Further, there is the possibility to add comments. The comments here are written to explain the feature tree figure.

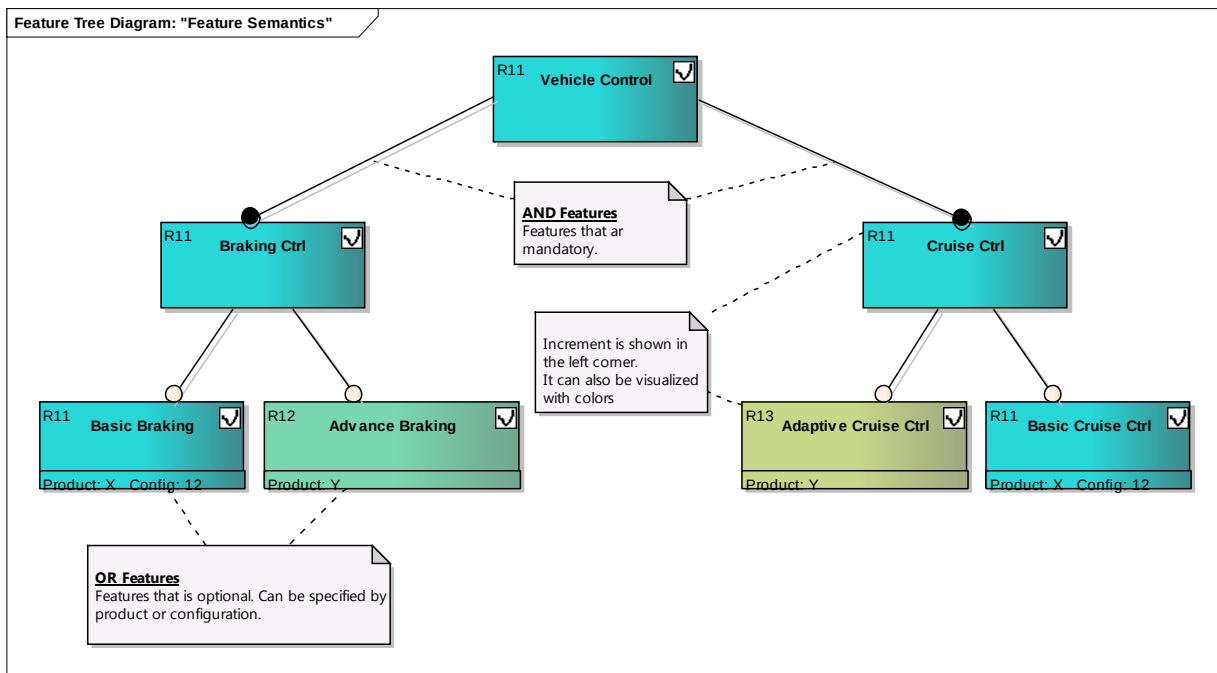


Figure 27 Feature tree decorated with steps of incremental development

The feature tree may in complex systems be rather large. Application of a filter to highlight relevant information and gray out other information may make the feature tree well understandable also in the case of a complex system. An illustration of applying a filter is given in Figure 28.

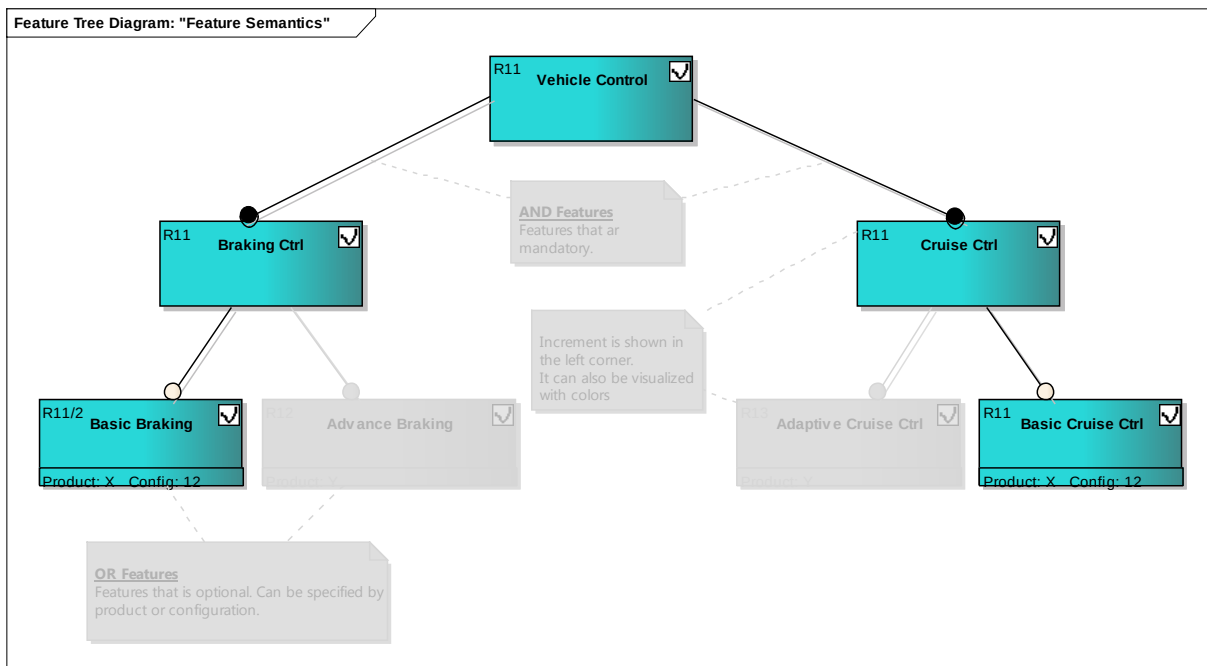


Figure 28 Feature tree with a filter to highlight desired information and gray out other

6.6 Features and Requirements

To understand a feature model for a system, the feature tree may be used in conjunction with requirements, grouped based on features, as illustrated in Figure 29. This would help to understand the feature model through the explanation of the requirements, and it would help to understand the requirements by seeing for which features they are relevant.

The view of a feature tree with requirements grouped on relevant features contributes to the scope as defined for the report in Section 2.2.1. This is a preferred view for showing requirements grouped on features.

Similarly to the Increments of Features view described in Section 6.6, the preferred concrete view is categorized as a tree view.

6.6.1 Example View

Figure 29 below shows features and their requirements according to Satisfy relationships. It is a viewpoint that does not scale to realistic sets of requirements, but filtering (showing only a subset of requirements) is a way to keep the number of requirements in the view manageable. Such views are also a good entry point into a model: Once the overall picture is communicated and understood through a graphical view, a tree view or list view can be used for the full scale work.

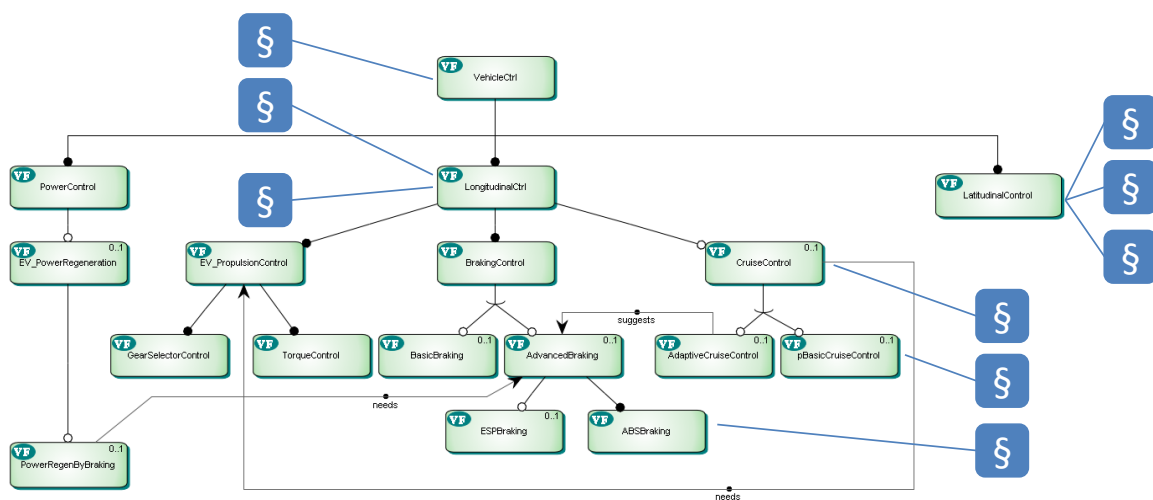


Figure 29 Feature tree decorated with requirements grouped on the relevant features

6.7 Functions and Requirements

To understand functions and their requirements, a view that illustrates (a subset of) the functions and the information flow between functions can be decorated with requirements grouped on the relevant functions. An illustration is given in Figure 30.

The view that shows requirements grouped on functions contributes to the scope of this report as defined in Section 2.2.1.

The preferred concrete view is categorized as a component view.

6.7.1 Example View

Figure 30 below shows functions and their requirements according to Satisfy relationships. It is a viewpoint that does not scale to realistic sets of requirements, but filtering (showing only a subset of requirements) is a way to keep the number of requirements in the view manageable. Such views are also a good entry point into a model: Once the overall picture is communicated and understood through a graphical view, a tree view or list view can be used for the full scale work.

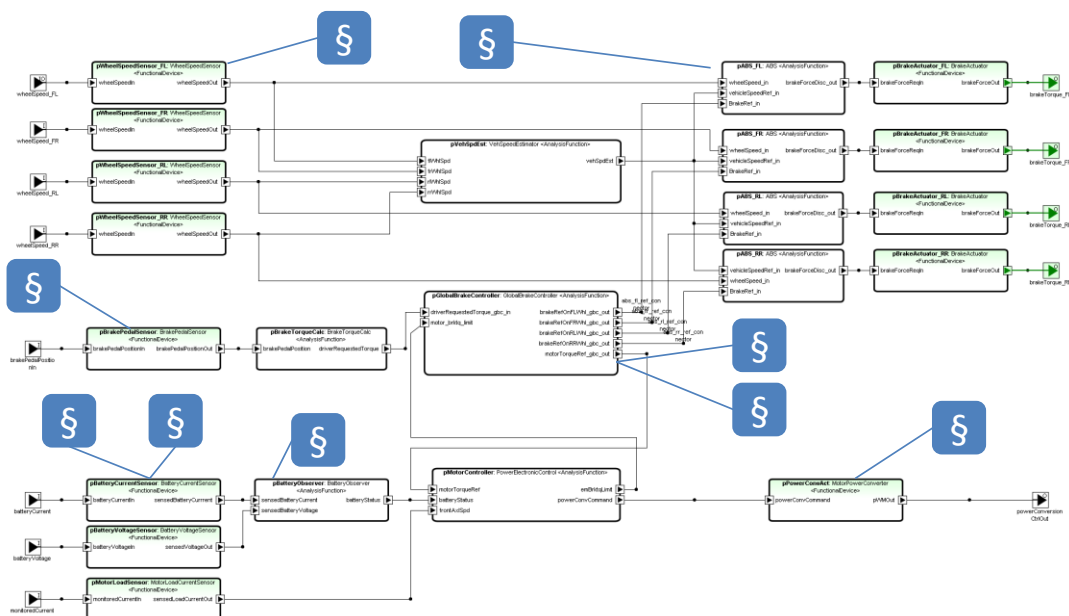


Figure 30 Functions with information flow, decorated with requirements grouped based on the relevant functions

6.8 Hardware Components and Requirements

To understand the modelling of the system on a hardware level and the corresponding requirements, a useful view is to show the hardware components and their signals in a schematic, and then decorate that schematic with requirements grouped on the hardware components that the requirements are allocated to. Figure 31 illustrates the view.

This view for showing requirements allocated on components contributes to the scope of the report, see Section 2.2.1.

The preferred concrete view is categorized as a component view.

6.8.1 Example View

Figure 31 shows hardware components and their requirements according to Satisfy relationships. It is a viewpoint that does not scale to realistic sets of requirements, but filtering (showing only a subset of requirements) is a way to keep the number of requirements in the view manageable. Such views are also a good entry point into a model: Once the overall picture is communicated and understood through a graphical view, a tree view or list view can be used for the full scale work.

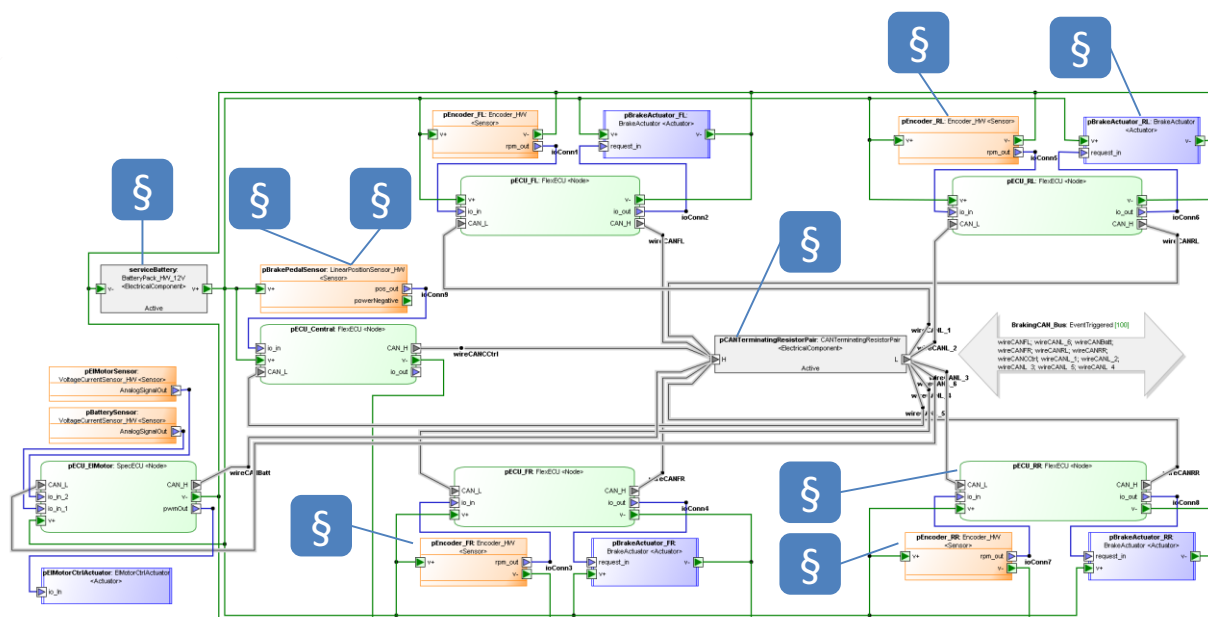


Figure 31 Hardware components and electrical signals in a schematic, decorated with requirements allocated to the hardware components and signals

6.9 Allocation

To understand which functions are allocated to what component of the hardware architecture, a useful view is the table that shows the mapping between components of the software architecture (in rows) and functions (in columns).

This view contributes to the scope of the report (see Section 2.2.1) as a way to illustrate how functions are allocated onto the software architecture.

6.9.1 Example View

Figure 32 shows an example view, where hardware components in the columns/X axis are marked with a cross for each function (in the rows/Y-axis) allocated to it with FunctionAllocation relation.

AllocationMatrix: pAllocation_Braking_Config1, 7 oktober 2013, 15:32

	pBatterySensor_VoltageCur	pBrakeActuator_FL_BrakeAct	pBrakeActuator_FR_BrakeAct	pBrakeActuator_RL_BrakeAct	pBrakeActuator_RR_BrakeAct	pBrakePedalSensor_Linear	pECU_Central_FlexECU<N>	pECU_ElMotor_SpecECU<N>	pECU_FL	FunctionAll
pABS_FL_ABS										
pABS_FR_ABS										
pABS_RL_ABS										
pABS_RR_ABS										
pBatteryCurrentDM_Bat										
pBatteryCurrentSensor_B	FunctionAllocation									FunctionAllocation
pBatteryObserver_Battery										
pBatteryVoltageDM_Bat										
pBatteryVoltageSensor_B	FunctionAllocation									FunctionAllocation
pBrakeActuatorDevice_FL		FunctionAllocation								
pBrakeActuatorDevice_FR			FunctionAllocation							
pBrakeActuatorDevice_RL				FunctionAllocation						
pBrakeActuatorDevice_RR					FunctionAllocation					
pBrakePedalIO_BrakePed						FunctionAllocation				
pBrakePedalDM_BrakePh						FunctionAllocation				
pBrakePedalSensor_Peda						FunctionAllocation				
pBrakeTorqueMap_BrakeT							FunctionAllocation			
pGlobalBrakeController								FunctionAllocation		
pHW_Brake_FL_BrakeAct		FunctionAllocation								
pHW_Brake_FR_BrakeAct			FunctionAllocation							
pHW_Brake_RL_BrakeAct				FunctionAllocation						
pHW_Brake_RR_BrakeAct					FunctionAllocation					
pHW_Encoder_FL_WhiRot										
pHW_Encoder_FR_WhiRot										
pHW_Encoder_RL_WhiRot										
pHW_Encoder_RR_WhiRot										
pLoadCurrentDM_LoadC										
pLoadCurrentSensor_Moto										
pPowerConvActuator_Pow										
pPowerConvActuatorLDM										
pPowerEICtrl_PowerEICtrl										
pVehicleSpeedEstimator										
pWhiSpeedSensorDevice							FunctionAllocation			
pWhiSpeedSensorDevice								FunctionAllocation		
pWhiSpeedSensorDevice									FunctionAllocation	
pWhiSpeedSensorDevice										FunctionAllocation

Figure 32 Functions allocated to nodes in a distributed system

6.9.2 Related Modelling Concepts

Figure 33 shows elements relevant for showing in an allocation view. The Specializations of design functions and hardware components can also be used, but are omitted in the figure.

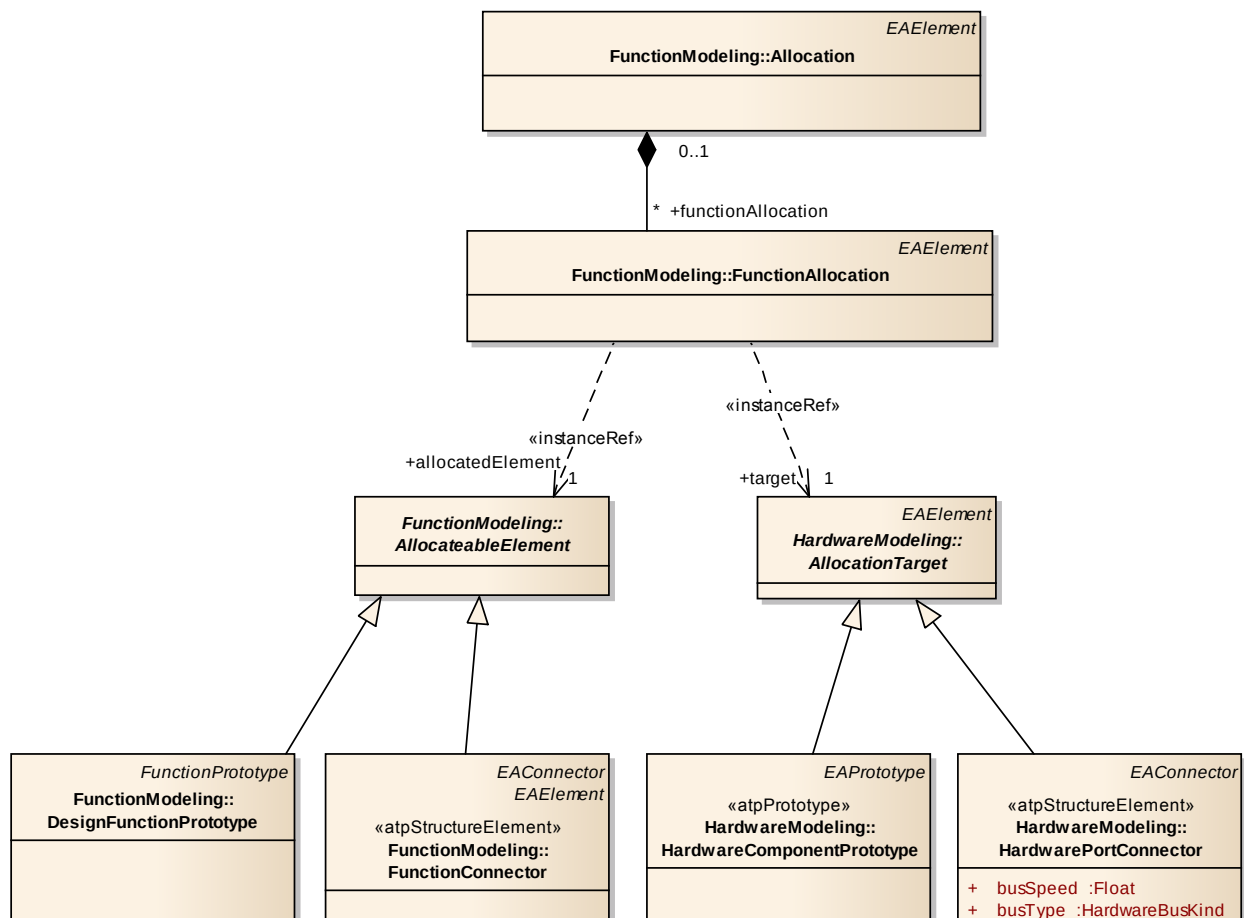


Figure 33. Allocation targets and subjects

6.10 Feature-Based Architecture Views

These views show the System and Software Architecture with regard to elements that realize a certain feature. This contributes to the scope of the report as defined in Section 2.2.1.

Architectural elements, such as functions, sensors or ECUs have the responsibility to realize one or several Vehicle Features. To identify which architectural elements are related to a certain Vehicle Feature, it is useful to have a view that hides other elements or highlights the relevant elements.

The preferred concrete view is categorized as a composite concrete view of a tree view and a component view.

6.10.1 Example View

Figure 34 shows how elements that realize a selected feature are highlighted.

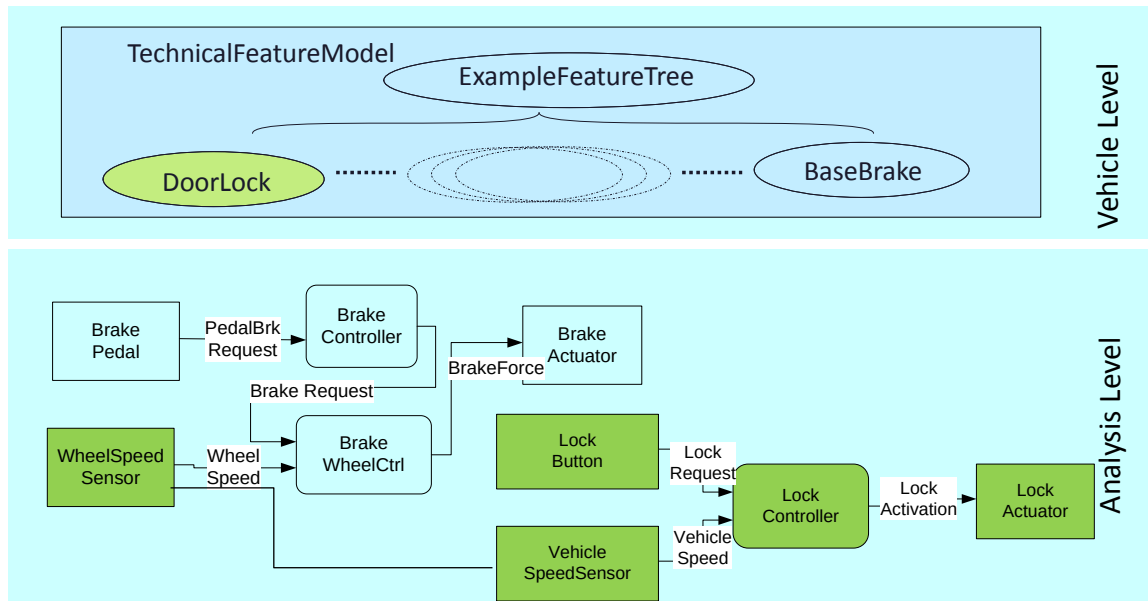


Figure 34 Highlighting elements that realize a specific feature.

6.10.2 Related Modeling Concepts

The elements that realize a specific feature are found by following the realize relations from the feature to elements on the analysis, design or implementation level. This contributes to the scope of the report as defined in Section 2.2.1.

Figure 35 shows the relevant metamodel excerpt.

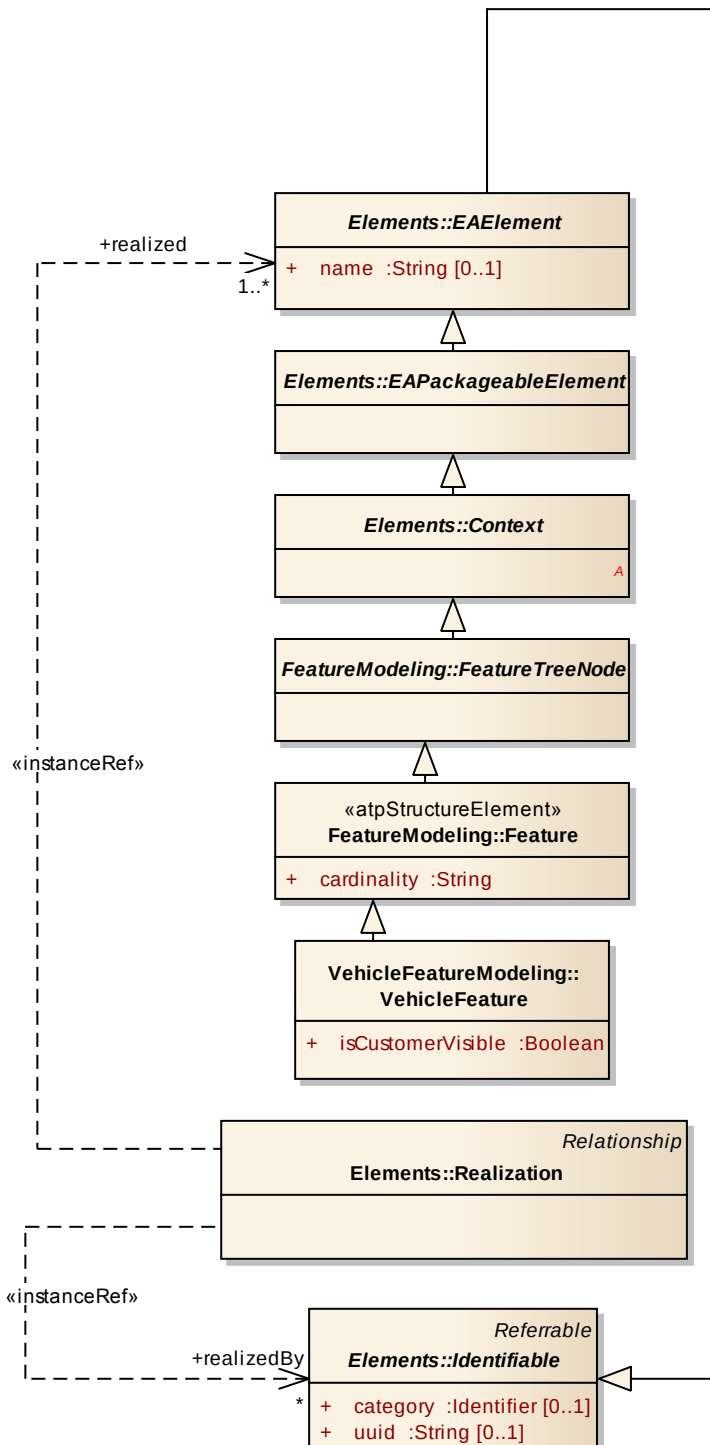


Figure 35  Realization relation between Identifiable and EAElement.

6.11 Requirements vs. Architecture

Requirements are normally related to specific parts of the functional, hardware and software architecture. It is expected that the allocation of requirements can make the requirement set easier to understand and manage. Views that present the requirements according to the system hierarchy are thus beneficial.

This view shows relevant attributes of the requirements. Furthermore, requirements are selected to be relevant for the functions and system modelling components that are shown in the view (see the example view below). In these two aspects, this view contributes to the scope of the report as defined in Section 2.2.1.

The preferred concrete view is categorized as a composite concrete view of multiple component views.

6.11.1 Example View

Figure 36 shows requirements as they are allocated to architectural components.

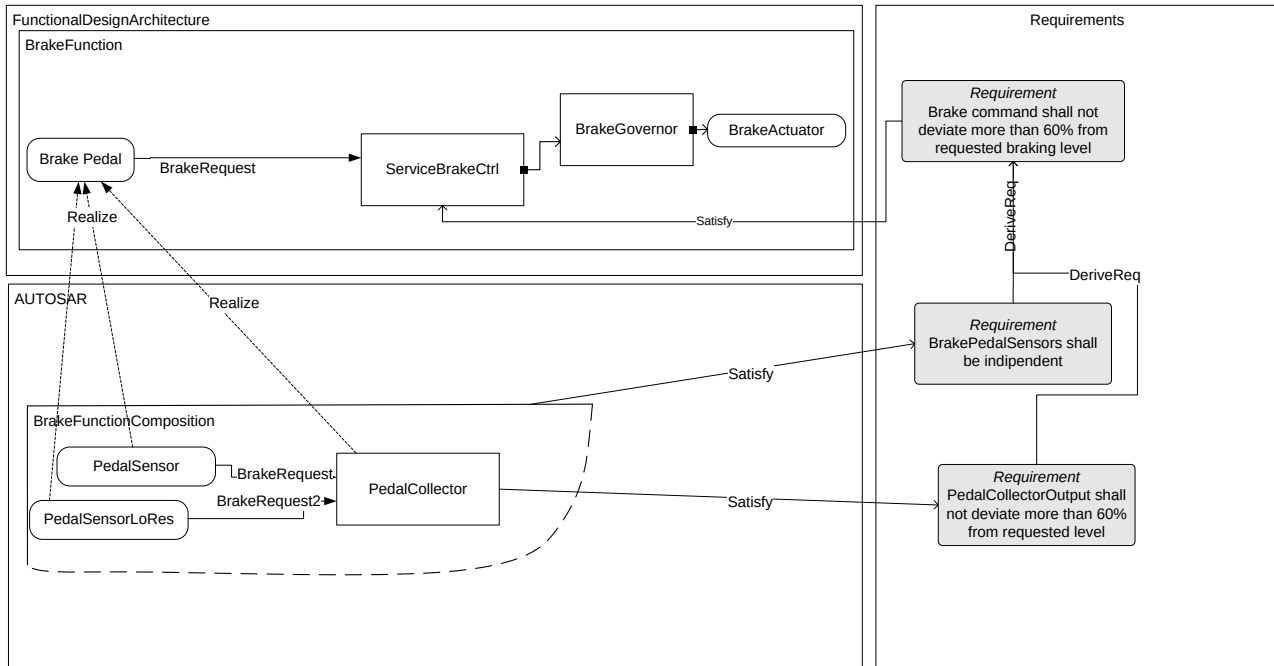


Figure 36 Example of requirements and architectural elements shown together.

6.11.2 Related Modelling Concepts

The Requirement-Architecture viewpoint relies on requirements (Requirement), architectural elements (Identifiable) and the Satisfy relation. Figure 13 shows how these elements relate.

6.12 Components, Ports and Signals

This view is there to show the architecture of distributed software. In doing so, it contributes to the scope of the report as defined in Section 2.2.1.

On the analysis, design and implementation level, functional and software components (for example a set of communicating applications on an AUTOSAR platform) are in some activities modelled as components with ports and connectors to represent connectivity and how data may flow. With many components, many signals and various types of ports (sender/receiver, client/server, modes) the complexity easily increases, leading to a system that is difficult to make sense of, unless something is done to provide overview.

The preferred concrete view is categorized as a component view.

6.12.1 Example Views

Depending on the size of the model the information shown by a view could have varying degrees of advantages. A small model is much easier to process than a large one. Furthermore, the larger the model the more information the user would be have to process and too large a model might be disadvantageous rather than providing any benefits. Figure 37 attempts to illustrate the main issue with providing a view which illustrates a fairly large model. Although the view gives all the information the user might need in order to understand the flow of signals throughout the model the sheer amount of input one has to process makes the view nearly impossible to understand. The connections alone are so abundant they obscure whatever useful information one might be able to gain from such a view. Certainly, information overflow is an issue here and it should be beneficial to limit the input provided to the view or give alternatives in how to view the model.

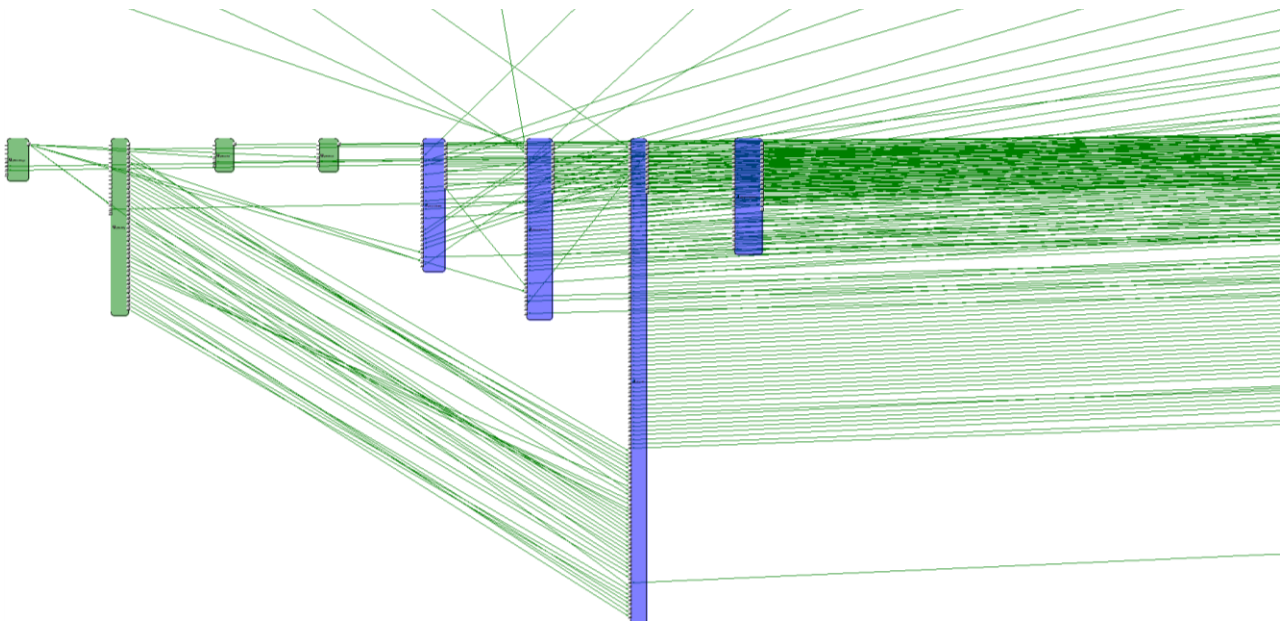


Figure 37  Large example of Components and Connections

Figure 38 demonstrates a situation which is much simpler to process. Here the components are not as large and the number of connections is rather limited. Despite many connections crossing components and other connections it is still fairly simple to gain an understanding of how the signals flow through the model. Obviously, not all models are as small as this one, Figure 37 clearly demonstrated a more complex System.

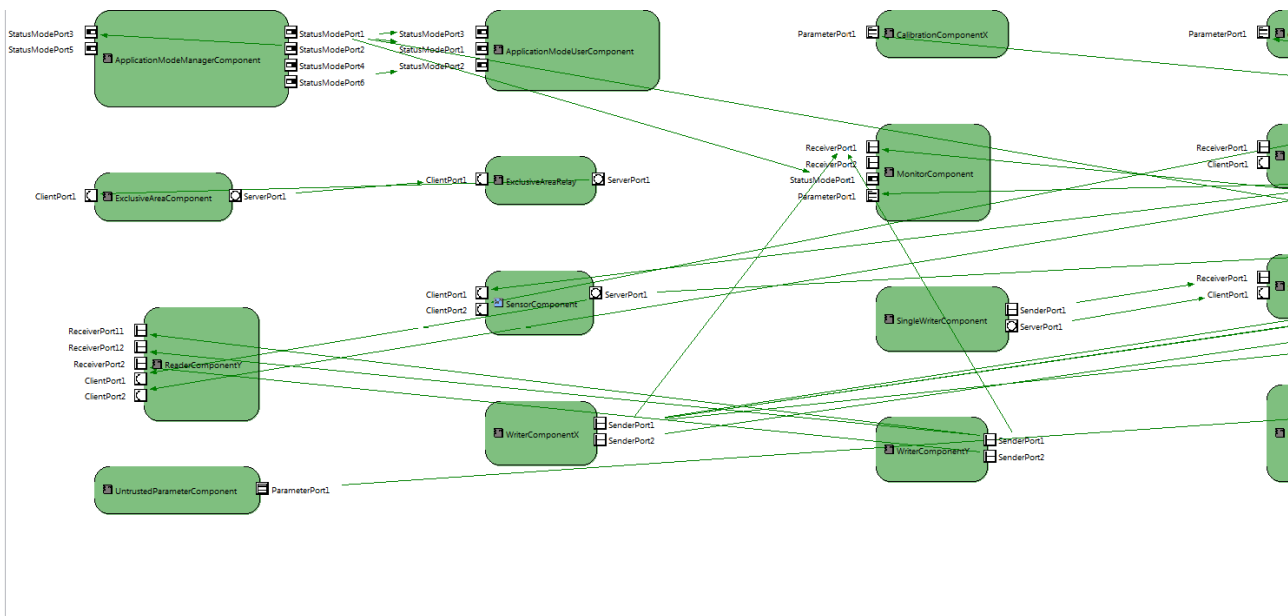


Figure 38 Small example of Components and Connections

Consequently, here is where considerations could be made to improve how input is handled and limit the parts of the model shown by the view. Consider for example if one would only show a small part of the whole model demonstrated in Figure 37, perhaps the input to the view could be only a few select components which are relevant at the time. The user could specify more precisely which parts of the system which should be shown and the view would only render those. Certainly, such improvements would simplify the generated view and make the handling of such a view a simpler matter for whoever needs to view it.

Furthermore, it might be beneficial to add usability tools to the view which could enable to user to obscure information, such as connections or ports which is at the time irrelevant. Figure 39 demonstrates a more focused view of that same example shown in Figure 37, here all irrelevant connections have been hidden, and the important component for this case has been put into focus.

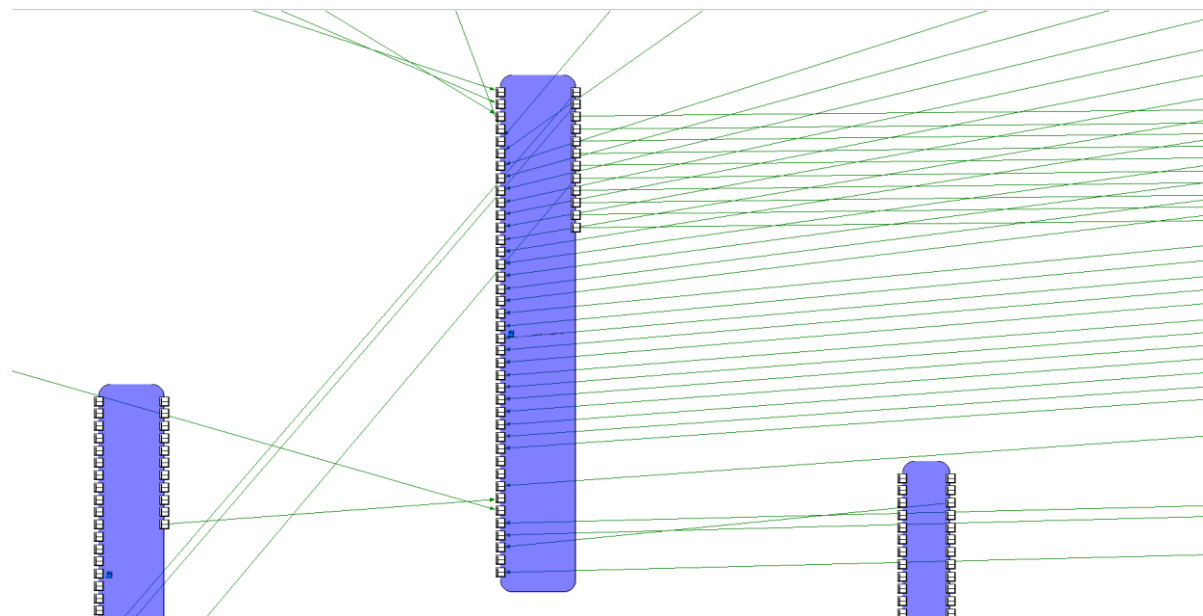


Figure 39 Focused view of large example in Figure 37

Even further tools could be provided to the user which could enable filtering away unnecessary information. Imagine if the user could simply open up a search window and type in a couple of key words and the view would then remove any components, ports and signals which do not match the result. It is important that it is context sensitive; certain restraints would have to be put in place in order to not obscure too much information. For example, if a user searches for a unique component name it would be disadvantageous to show only that component and hide everything else. Understandably, every connection and port associated with that component should also be shown.

A final mechanism to consider is the use of bus lines and goto blocks like in Simulink. The EAST-ADL concept of port group is useful to deduce which connectors can be combined to a single, thick line: All connectors from a certain port group can be combined, and only split up when approaching destination ports and port groups. Goto blocks are not supported by EAST-ADL, but are possible diagram functionality in a specific tool: Instead of lines between ports, the connector is represented by lines plus goto blocks to avoid long and crossing lines.

6.12.2 Related Modeling Concepts

EAST-ADL contains the modeling elements for components, ports and signals. However, EAST-ADL does not specify the information elements to hold the necessary information to show the view. For example, EAST-ADL has no attributes for the position and size of a rectangle for the component. To show a concrete (decorated) view, information that dictates the visual appearance of the view has to be represented using some other information than EAST-ADL. The introduction of a structured information model for diagram information is thoroughly discussed in Section 3.11.

AUTOSAR can hold more concrete information of how a system can be modeled on an implementation level. AUTOSAR contains meta-model elements which can represent concrete components, connections and ports. Components hold ports which can be interconnected with other components via connections forming AUTOSAR applications. The AUTOSAR SW component template represents a component hierarchy that is a suitable basis for the described view. As for EAST-ADL the graphical information is missing in the AUTOSAR model, but can be stored in the same complementary structure.

6.13 Item Definition

ISO26262 use the concept of Item to define the “system or array of systems to implement a function at the vehicle level, to which ISO 26262 is applied”. Elements of an Item are hardware (ECUs, sensors, cables etc.) and software (functions, connectors, software components, etc.) of various kinds and represented on various abstraction levels. During Item definition it is therefore useful to show the elements concerned to secure the right boundary. Further, it is useful to show the safety goals for the item, as they are defined.

A view to show the safety goals for a certain item contributes to the scope of this report defined in Section 2.2.1.

In a FunctionalDesignArchitecture, HardwareDesignArchitecture or SoftwareComponentTemplate, it is thus useful to hide all elements that are not included in the Item that is inspected.

The preferred concrete view is categorized as a composite concrete view of multiple component views.

6.13.1 Example View

Figure 40 shows an example of how Features and Functions relate to Item and gives an idea about how an Item subset of a FeatureTree or Functional Architecture can be derived.

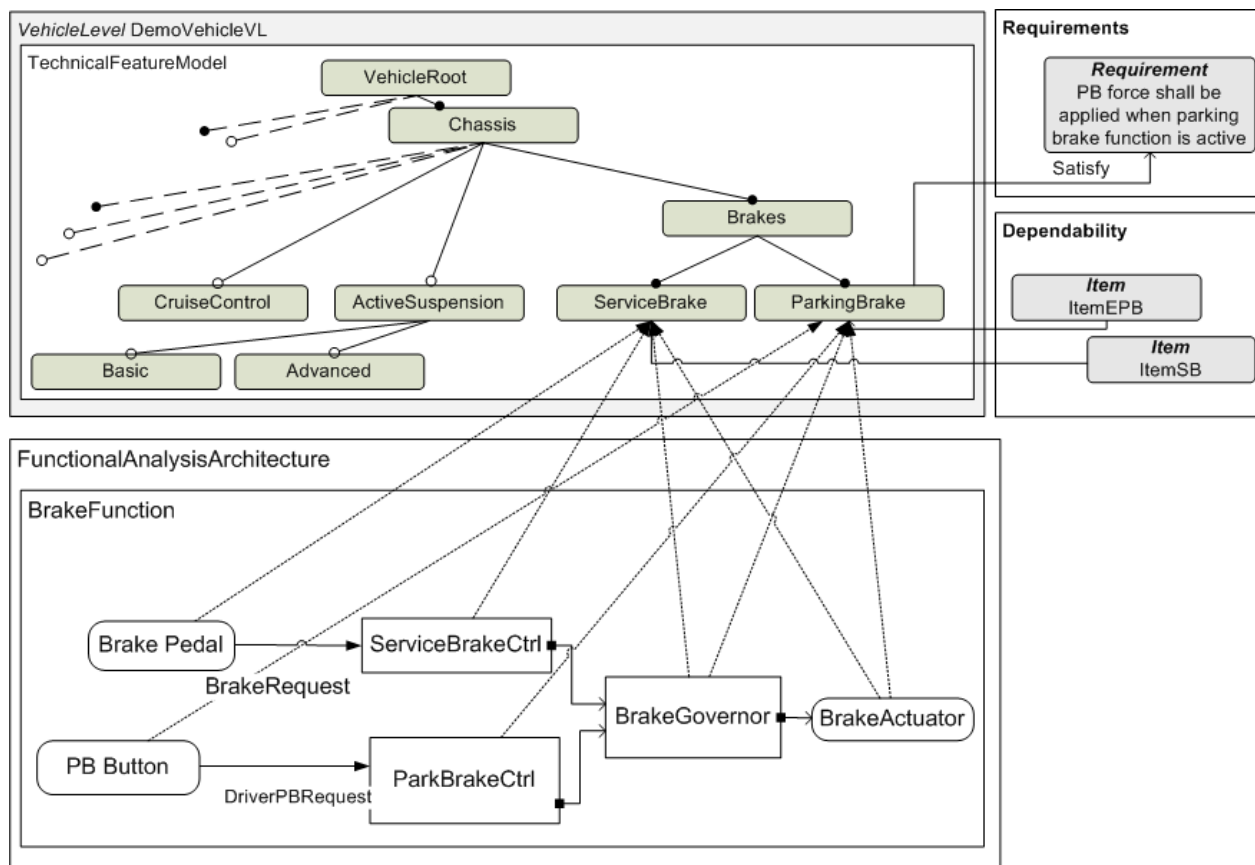


Figure 40 Example of an Item Definition View

6.13.2 Related Modeling Concepts

Figure 41 shows EAST-ADL model elements relevant for the view.

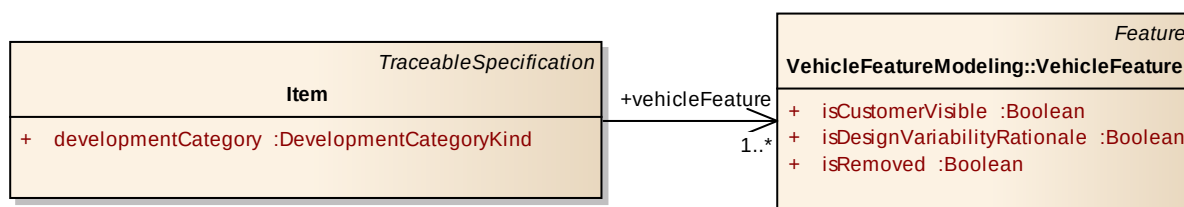


Figure 41 Item and its reference to one or several VehicleFeatures

The Item element references VehicleFeatures, which in turn are referenced by all elements that realize the feature. This means that any EAST-ADL or AUTOSAR element may be included in an item. Figure 42 shows how an identifiable (such as an AUTOSAR SWC or EAST-ADL Function) realizes an EAElement (such as Feature or Function).

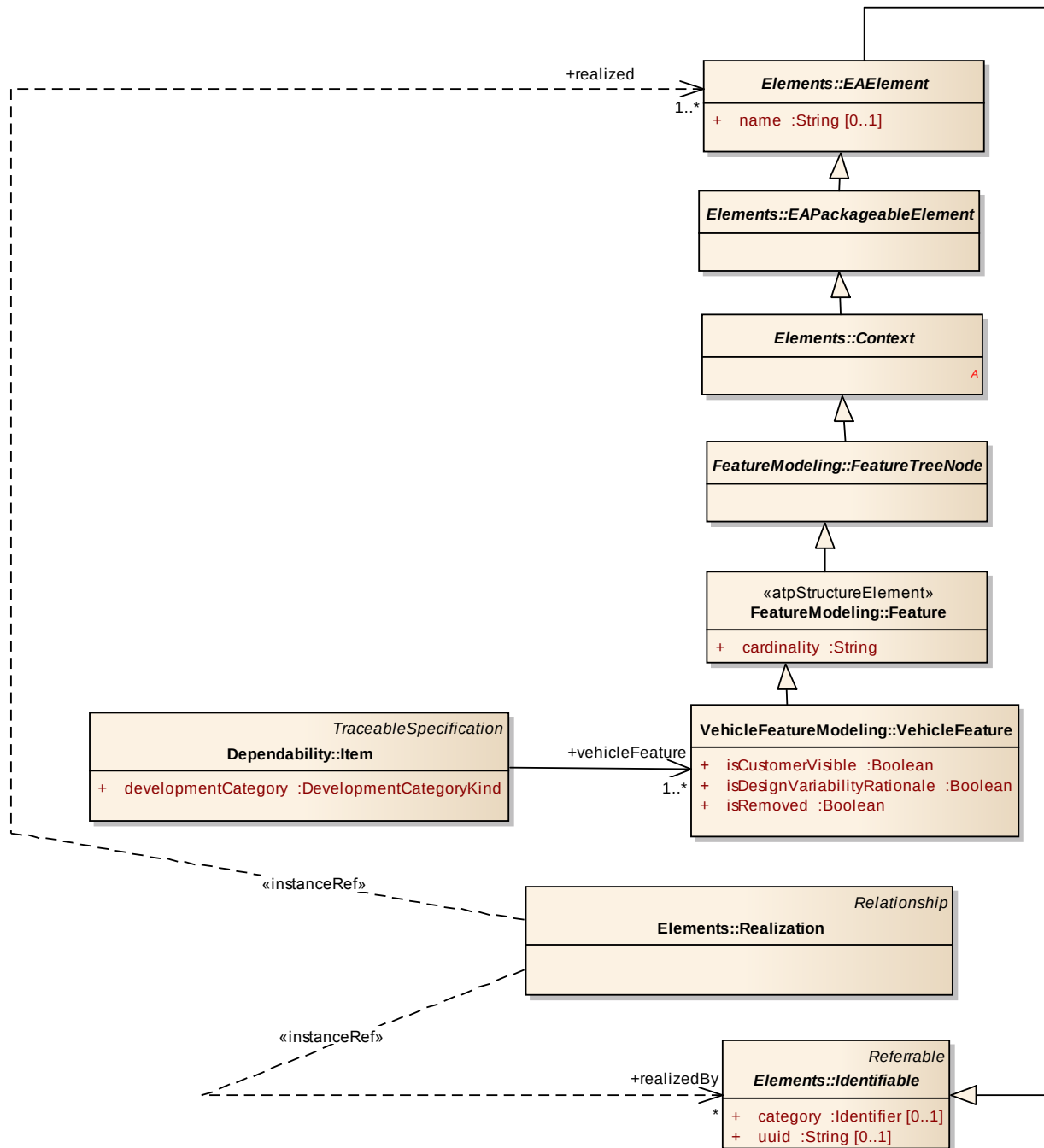


Figure 42 ■ Realization relation between realized Element and realizing identifiable.

6.14 Functional and Technical Safety Concept

Visualizing the Functional Safety Concept or the Technical Safety Concept is important to assess what measures are taken to meet the safety goal. Functional and Technical Safety Concepts are composed of sets of safety requirements allocated to architectural elements.

Views for illustrating the part of the system model that correspond to the functional safety concept and the technical safety concept are contributing to the scope of this report as defined in Section 2.2.1.

The preferred concrete view is categorized as a composite concrete view of multiple component views.

6.14.1 Example View

Figure 43 shows how the functional safety concept may be visualized. In the upper part, its Item elements are also included, although they are not part of the functional safety concept.

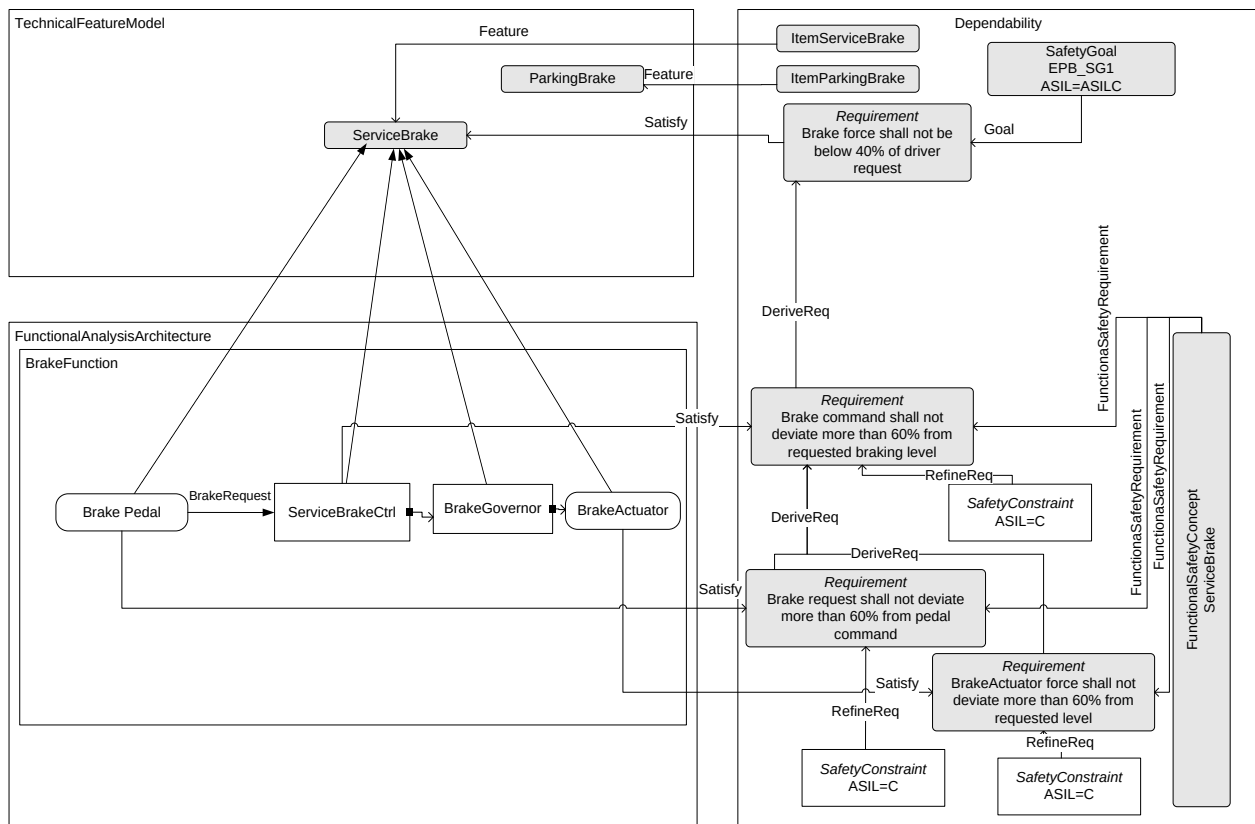


Figure 43 Functional Safety Concept

6.14.2 Modelling Concepts

The views of Functional Safety Concept and the Technical Safety Concept are based on the elements with the same names. Figure 17 above shows the elements and relations.

6.15 Error Propagation Modelling

Error propagation models have the purpose to define for each component (functional or physical, concrete or abstract) the expected internal faults and expected faults from adjacent components, the error propagation logic and the resulting potential failures of the component. This is closely related to views to support safety analysis, since the information of possible error propagation leads to the construction of a fault tree on with fault tree analysis is performed.

A view to support error propagation analysis and a view that shows a fault tree contribute to the scope defined for this report in Section 2.2.1.

The preferred concrete view is categorized as a composite concrete view of multiple component views.

6.15.1 Example View

Figure 44 shows a nominal model with one plausible error propagation model. The safety engineer may adjust the error propagation model further, which is why they are separated as two structures.

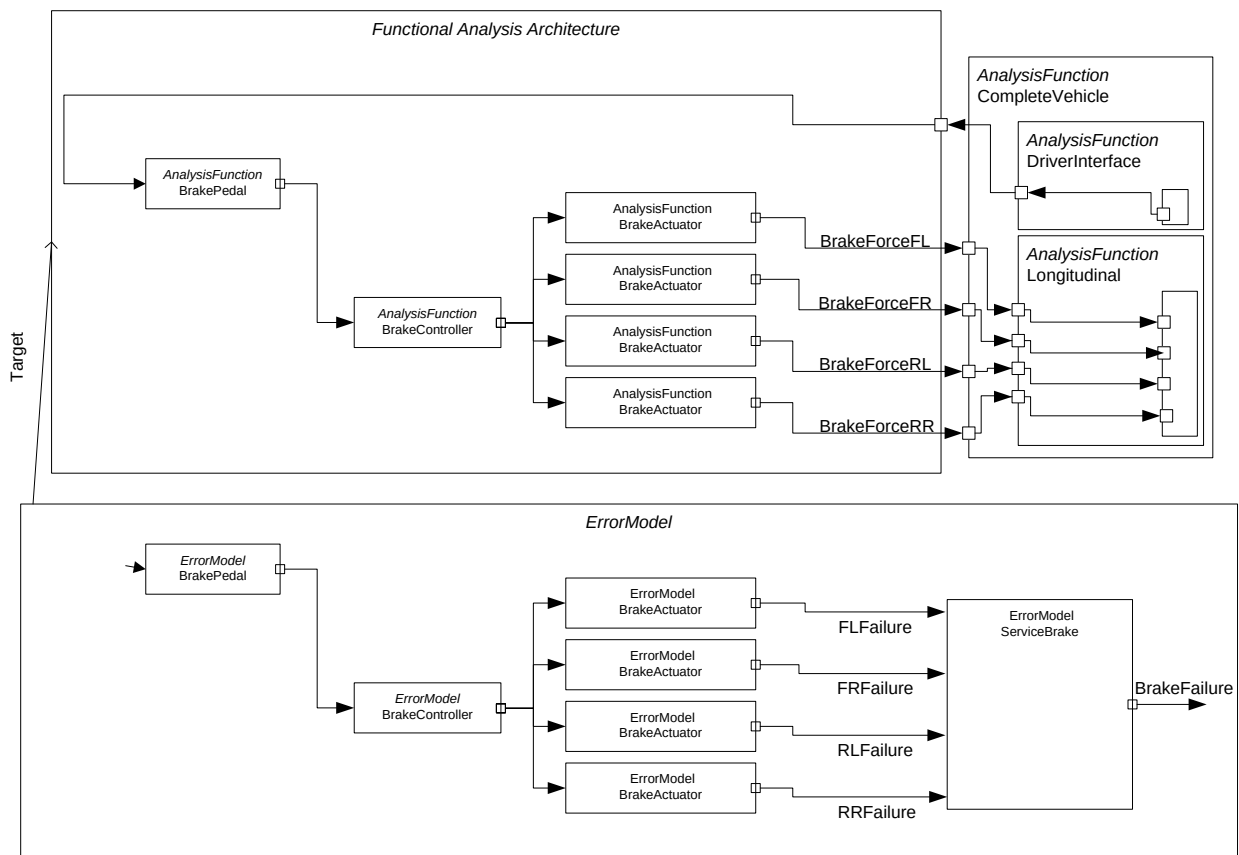


Figure 44 Example of nominal model (top) and its corresponding error propagation model

Another view of the error propagation model is to consider the failure logic of each block and derive a Fault Tree, see Figure 45



Error propagation is modeled using the `ErrorModelType` hierarchy, as described in Figure 46.



67 (73)

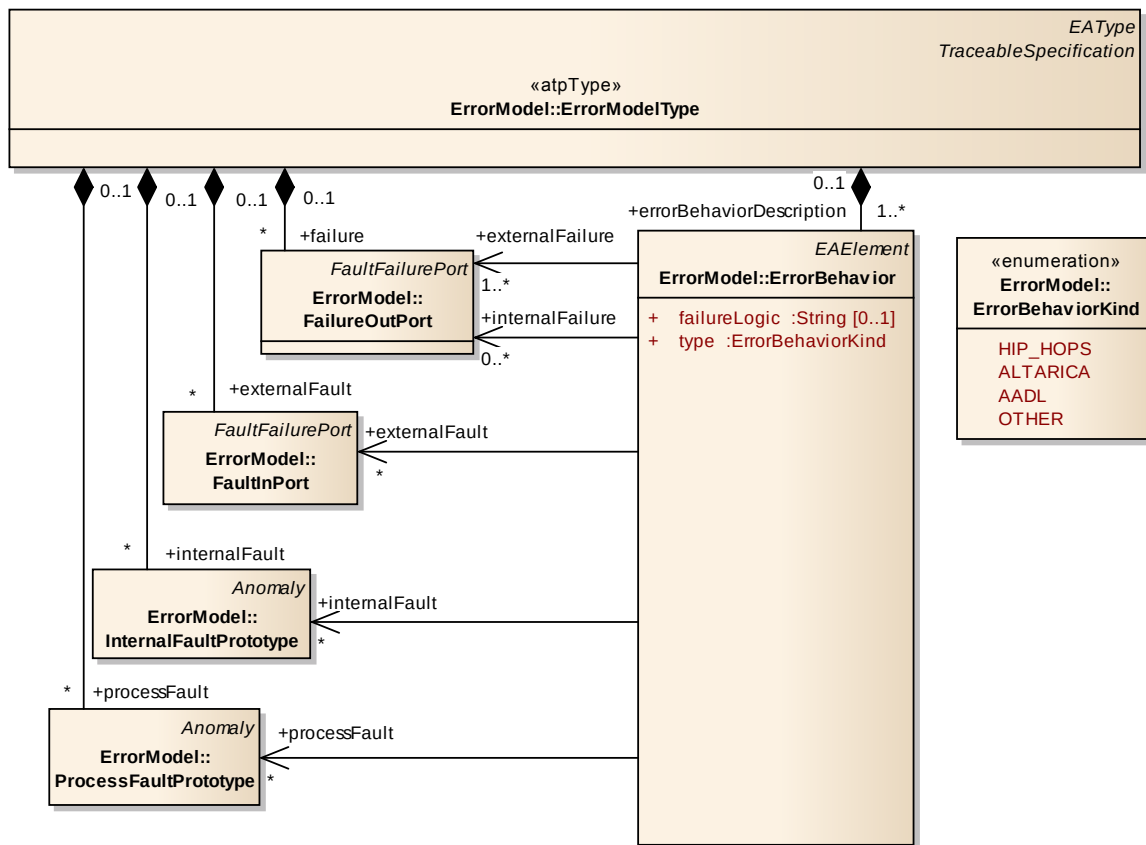


Figure 47 ErrorBehavior with failureLogic is the basis for defining error propagations.

The Error Propagation Model and the related Fault Tree Analysis are two views used to perform analysis which generates artefacts for the Functional Safety Concept and for the Technical Safety Concept.

6.15.3 Work-Phases when Eliciting Error Propagation Models

Working with Error Propagation models can be considered in four phases: Defining, Analysing, Understanding and Evolving.

6.15.3.1 Defining

The first step, defining, is to create the error propagation model from the nominal architecture model. Decisions are needed to determine which or what level of components are going to be used as a basis for error propagation model. There are also engineering decisions needed in order to define an efficient error propagation model, for example by identifying the right granularity. One or several Error Model components may capture the error propagation behaviour for a part of the architecture. The kind of errors considered will also be a critical engineering decision. While a lot of engineering decisions are involved, synthesis and microautomation support from the tooling is important for efficiency.

A feasible approach to assist error propagation modelling is to start with a 1-1 auto-generation of an error propagation model based on the nominal architecture. Because error propagations may occur beyond direct inter-component links, a manual effort is required to modify the error propagation model. Similarly, where multiple nominal links connect two components, their error propagation characteristics may be captured by a single link and a set of nominal components may be represented by a single error propagation component. For these reasons, collapsing of components and ports need to be done and tool assistance is good.

6.15.3.2 Analysing

The second step is the analysing process, by means of fault tree analysis [5]. In this step, the fault propagation data in the error model would be synthesized and analysed into a fault tree. It should be noted that fault tree analysis is a wide-spread analysis method with a well-known view. For this reason, the view is not further described here. The analysis tool would automatically generate the corresponding fault tree once the error model is decided.

Different analysis tools and notations (AltaRica, HiP-HOPS, Rodelica, AADL) are available, and the failure logic must be defined accordingly.

6.15.3.3 Understanding

To understand the generated error propagation model and see its relation to a nominal model, efficient views are needed. This relates both to the error propagation model, and the corresponding fault tree or FMEA table. By visualizing failures, failure propagation paths, etc. it is possible to see (c.f. Figure 44) how the system elements relates to the identified faults and error propagations.

6.15.3.4 Evolving

After the initial definition of models, and subsequent analysis, there will be adjustments of the nominal model and of the error propagation model. To secure consistency, modifications of the nominal model should result in the modification of the error model and vice versa.

6.16 Illustrating Metrics

It is useful to decorate various views of the system model with annotations of metrics, to see the status or progress status of system model components as well as their contribution to product metrics such as dependability. There are also views just to show metric values, often called a cockpit view.

Views that annotate metrics to system model illustrations or provide a range of metrics contribute to the scope of this report as defined in Section 2.2.1.

The preferred concrete view is categorized as a graph view. There is another concrete view, categorized as a component view, which can be useful for illustrating metrics.

6.16.1 Example View

Figure 48 shows an example of a cockpit view that illustrates profiling of six contributors to the subsystem's latency (left) and an overview of the relative number of requirements that have been rejected.

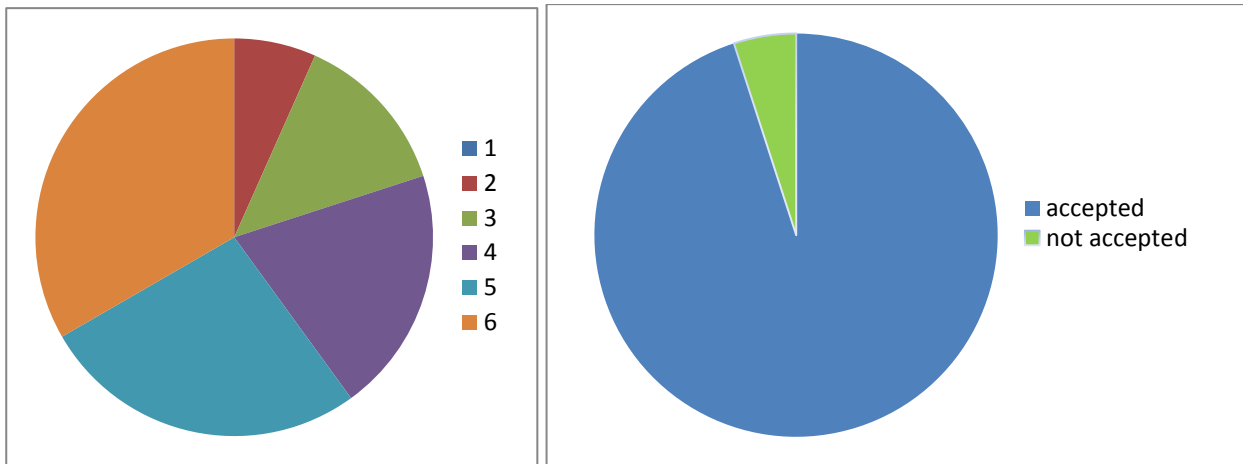


Figure 48 Aggregation of metrics in a cockpit view

Figure 49 shows how metrics can be used to give an overview of the progress on implementing the subsystem components. Green components have progress according to plan, while light-blue components are lagging behind.

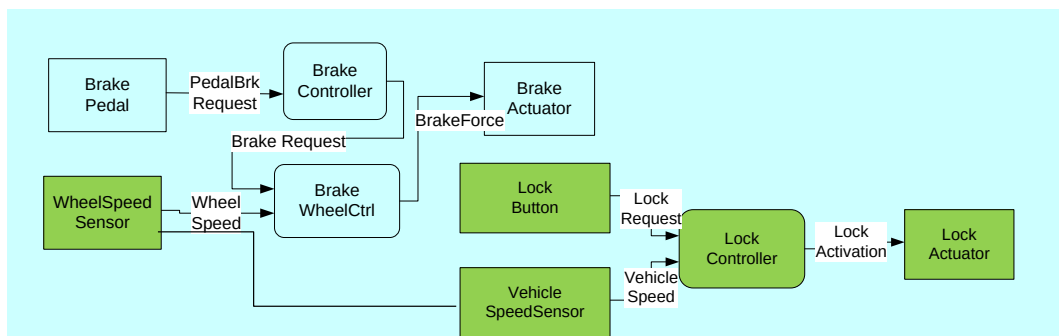


Figure 49 Alternative view for illustrating metrics

7 Summary

This chapter summarizes the outcomes of WP2.

The scope of the report has been detailed in Chapter 2, with a listing of what views and metrics that are mentioned in the report.

Information modelling concepts have been reviewed in Chapter 3. Four needs were mentioned, namely support for model distribution, simplification of version control, support for model exchange and model separation into architectural levels. EAST-ADL was reviewed and it was found that it provides model separation into architectural levels. However, EAST-ADL does not provide simplification of version control, since it does not have any dedicated property for version, time stamp or status. A model structure involving EAST-ADL, AUTOSAR and GraphML can be defined such that model parts can be handled as separate configuration items. This would be a step towards simplification of version control. In terms of model exchange, three scenarios with increasing complexity were defined. It was determined that Project Synligare should aim towards the second most complex scenario, namely a round-trip of model information without merge of models. Concerning support for model distribution, it was shown how Project Synligare is working with three tools, namely SystemWeaver, Enterprise Architect and EATop to enable various parties to work on different parts of a system model.

Concepts for prototype tools to show the views and metrics have been described in Chapter 3. In particular, on top of EAST-ADL, there are identified needs for additional information modelling for versioning information (Section 3.5) and diagram modelling (Section 3.11).

The interaction between OEM and Tier-1 in a collaborative development project has been analyzed in Chapter 4 to identify exchanges of information, with and without employing the benefits of model based development and common information models in Section 4.1 and Section 4.2 respectively. Metrics and views have been conceptually described, in Chapter 5 and Chapter 6 respectively, such that they can support the activities that are relevant to the interaction between OEM and Tier-1.

Chapter 5 lists the metrics that have been defined. They are of two main types, namely progress metrics and product metrics. The progress metrics concern requirements, verification, realization, and safety-related progress. The product metrics concern quality attributes like performance.

Chapter 6 lists the conceptual views that have resulted from WP2. The conceptual views can be categorized as tree view, table view, component view and graph view. Notable examples of views that are described include Compare and Merge, Increment of Features, views for requirements and modelling elements that they satisfy, Item Definition and Error Propagation Modeling.

8 Bibliography

- [1] Synligare Consortium, "Synligare Deliverable D1.1 Needs Identification," Volvo Technology AB, Gothenburg, 2014.
- [2] EAST-ADL Association, "EAST-ADL Schema eastadl_2-1-12.xsd," [Online]. Available: <http://www.east-adl.info/>. [Accessed 29 09 2014].
- [3] EAST-ADL Association, "Specification," [Online]. Available: www.east-adl.info. [Accessed 29 June 2014].
- [4] International Organization for Standardization, "ISO/FDIS 26262 Road vehicles -- Functional Safety, part 1 to 9," 2011.
- [5] C. Ericson, "Fault Tree Analysis - A History," in *Proceedings of the International System Safety Conference*, 1999.
- [6] AUTOSAR Development Partnership, "AUTOSAR Specification," [Online]. Available: <http://www.autosar.org/>. [Accessed 29 09 2014].
- [7] AUTOSAR Tool Platform User Group, "AUTOSAR Tool Platform," [Online]. Available: <http://www.artop.org/>. [Accessed 29 09 2014].
- [8] EATOP Eclipse Open Source Project, "EAST-ADL Tool Platform," [Online]. Available: <http://www.eclipse.org/eatop>. [Accessed 29 09 2014].
- [9] Synligare Consortium, "Synligare Project Website," [Online]. Available: <http://www.synligare.eu/>. [Accessed 29 09 2014].
- [10] Synligare Consortium, "Synligare Deliverable D4.1 Validation," 2014.

9 Acknowledgements

Figures in this deliverable were based on diagrams in EnterpriseArchitect from Sparx, MetaEdit+ from MetaCase as well as various tools and views developed by the Synligare partners.